

```
POST /orders
{"quantity": -5}

→ 201 Created

total_charged: -10000
```

THE COMPLETE GUIDE · HACKERSMANG 2026

Break My API

Can AI find the bugs
I forgot to test?

FAIL quantity: -5

FAIL role: admin

???? 20 buyers, 5 seats

PASS happy path

From "what is an API?" to running an AI adversarial-testing pipeline against a deliberately broken FastAPI service, reading the evidence honestly, and presenting it on stage.

Vidyarathna B

Software Engineer · Soult Digital

AI Talks track · Saturday 19 September 2026
10:00–10:30 IST · github.com/vidyarathna/break-my-api

[START HERE](#)

How to use this guide

This guide is written for two readers at once: someone meeting APIs and AI testing for the first time, and the speaker who has to explain it to a room in 30 minutes.

The one sentence to remember

CORE IDEA

AI generates testing hypotheses. Execution provides evidence. Humans decide whether the behaviour is a defect.

The model is a fast source of *guesses*. A runner turns guesses into *evidence*. A person who knows the system turns evidence into a *decision*. Skip any one and the technique becomes useless or misleading.

Five parts, one path

PART 1

Foundations
APIs · tests · LLMs

PART 2

The project
code · 10 bugs

PART 3

AI pipeline
prompts · runs

PART 4

Applying it
CI · real teams

PART 5

Presenting
script · Q&A

Pick your reading path

If you are...	Read	You will be able to...
New to APIs or testing	Part 1 fully, then chapters 8–9 and 17	Explain what the talk is about to a friend.
A developer or tester	Skim Part 1, read Parts 2–4	Run the pipeline against your own API.
The speaker	Parts 2, 3 and 5; keep the checklists open	Deliver a 30-minute talk with a demo that can't embarrass you.
A reviewer or organiser	Chapters 7, 17–19 and 26	Judge whether the claims are supported.

A NOTE ON HONESTY

Part 3's numbers come from real runs of a local model (`qwen2.5-coder:7b` , via Ollama) on 17 Sep 2026: one model, one small API, a few runs. A worked example, not a benchmark.

Contents

Part 5 was rewritten on 18 Sep 2026 for the HackersMang session, the 26-slide deck and the latest AI runs.

START HERE

How to use this guide	2
-----------------------	---

PART 1 · FOUNDATIONS

1. What is an API?	4
1.1 Anatomy of an HTTP request	5
2. Status codes and JSON	6
3. What is software testing?	7
4. Why bugs slip through	8
5. Testing techniques compared	9
6. What an LLM is, simply	10
7. The big idea: AI as an adversary	11

PART 2 · THE PROJECT

8. Project overview	12
9. Architecture	13
10. The ConfBadge domain	14
11. Endpoint reference	15
12. Code tour: authentication	16
12.1 Code tour: placing an order	17
12.2 Code tour: search and the database	18
13. The ten intentional bugs: map	19
B1–B10 · One page per bug	20
14. Running the project locally	30

PART 3 · THE AI PIPELINE

15. The pipeline, end to end	31
16. Prompt design principles	32
17. The four prompts	33

18. Inside generate.py and runner.py	35
19. Real run: Prompt 2	36
19.1 Real run: Prompt 3, twice	37
20. Triage: deciding what's a defect	38
21. A catalogue of AI mistakes	39
22. The honest scorecard	40
23. Getting better results	41

PART 4 · APPLYING IT

24. Using this on your own API	42
25. Putting it in CI/CD	43
26. Security, privacy and safety	44
27. Is it worth it? Measuring value	45
28. Fitting into your tools	46

PART 5 · TOMORROW'S SESSION (UPDATED 18 SEP)

29. Tomorrow's session and its theme	47
30. Your talk in one minute	48
31. Latest results and how efficient the AI was	49
32. Concepts you must be able to explain	50
32.1 AI and engineering concepts	51
33. The 30-minute plan (26 slides)	52
34. Slide by slide: what to explain on each	53
35. Demo runbook and backup plan	62
36. Q&A: what people will ask	63
36.1 Q&A, continued	64
36.2 Q&A, harder questions	65
37. Checklists for tonight and tomorrow	66

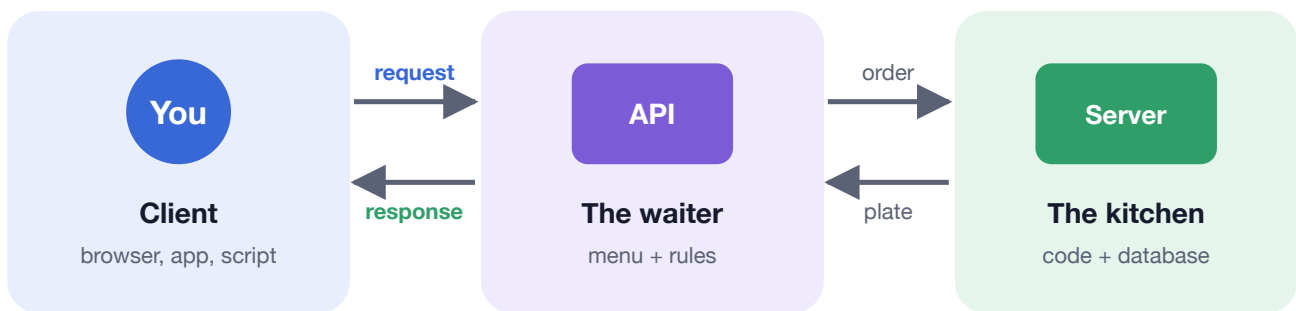
CHAPTER 1

What is an API?

An API is a menu of things one program lets another program ask for, and a promise about what comes back.

The restaurant analogy

You don't walk into a restaurant kitchen and start cooking. You read the menu, tell the waiter what you want, and the waiter brings back a plate. You never see how the kitchen works, and the kitchen never needs to know who you are beyond your order.



The client never touches the database directly. Everything goes through the API.

Figure 1.1 — A client talks to an API; the API talks to the code and data behind it.

Why this matters for testing

Because the API is the **only door** into the kitchen, it is also the only place where a bad order can be stopped. If the waiter accepts "minus five pizzas", the kitchen will try to cook minus five pizzas. Almost every bug in this guide is an API accepting an order it should have refused.

Web APIs

Speak HTTP over the network. This guide's project is one.

Library APIs

Functions you call inside a program, like Python's `json.loads`.

OS APIs

How programs ask the operating system for files, memory, network.

IN ONE LINE

An API is a contract. Testing an API means checking that the contract holds, especially when the caller does something the author never expected.

Anatomy of an HTTP request

A web API conversation is a request and a response. Both have the same three layers: a first line, headers, and an optional body.

REQUEST

POST /orders HTTP/1.1

Authorization: Bearer MTp1c2Vy

Content-Type: application/json

`{"ticket_id": 1, "quantity": 2}`

① method + path: what to do

② headers: who, what format

③ body: the actual data

RESPONSE

HTTP/1.1 **201 Created**

Content-Type: application/json

`{"order_id": 1, "total_charged": 4000.0,
"credits_earned": 400.0, "credit_balance": 1400.0}`

status: did it work?

body: the result

Figure 1.2 — A real request to this guide's demo API: Alice buys two tickets.

The four methods you'll see

Method	Meaning	Example here	Safe to repeat?
GET	Read something	GET /tickets?q=WORK	Yes: reading twice changes nothing
POST	Create or perform an action	POST /orders	No: two POSTs buy twice
PUT/PATCH	Replace or update	(not used)	PUT should be; PATCH depends
DELETE	Remove	(not used)	Should be

WHY "SAFE TO REPEAT" MATTERS

Networks retry and users double-click. Sending the same POST twice simulates a normal Tuesday, and it's exactly how this project's "single-use" coupon bug is found.

Try it yourself

```
# log in, keep the token, buy two tickets
TOKEN=$(curl -s -X POST localhost:8000/auth/login -H 'content-type: application/json' \
  -d '{"email":"alice@corp.example","password":"alice123"}' | jq -r .access_token)
curl -s -X POST localhost:8000/orders -H "authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"ticket_id":1,"quantity":2}'
```

CHAPTER 2

Status codes and JSON

The status code is the API's one-word verdict. JSON is the language the details are written in.

Status code families

2xx

It worked

200 OK

201 Created

Dangerous when it is a lie:
"201" for minus five tickets.

4xx

You asked wrong

400 Bad Request

401 Unauthorized

403 Forbidden · 404

409 Conflict · 422

Good: the API refused politely.

5xx

The server broke

500 Internal Error

503 Unavailable

A 500 caused by user input
is almost always a bug.

Figure 2.1 — The first digit tells you who is at fault.

THE TESTING TRICK

Most adversarial tests in this guide are just this question: **"When I send something wrong, do I get a 4xx?"** A 2xx means the API accepted garbage. A 5xx means it crashed on garbage. Both are findings.

JSON in five minutes

```
{
  "ticket_id": 1,           // number
  "quantity": 2,
  "coupon": null,          // "no value"
  "email": "a@b.com",      // string
  "paid": true,            // boolean
  "tags": ["vip"],         // array
  "owner": {"id": 9}       // object
}
```

JSON has only six kinds of value: **number, string, boolean, null, array, object**. That small list is exactly why it is so useful for attacking an API: for every field, you can ask "what if it were one of the other five?"

Field expects	Try sending
number 2	-5 , 0 , "2" , 2.5 , null
string	"" , 10 000 chars, ' OR 1=1
object	[] , {} , missing

Real JSON has no comments; they're added above for teaching. In this project, Pydantic rejects the wrong *types* automatically, but nothing rejects the wrong *values*, like `-5` . That gap is where most bugs live.

CHAPTER 3

What is software testing?

A test is a question you ask the code, with the answer you expect written down in advance.

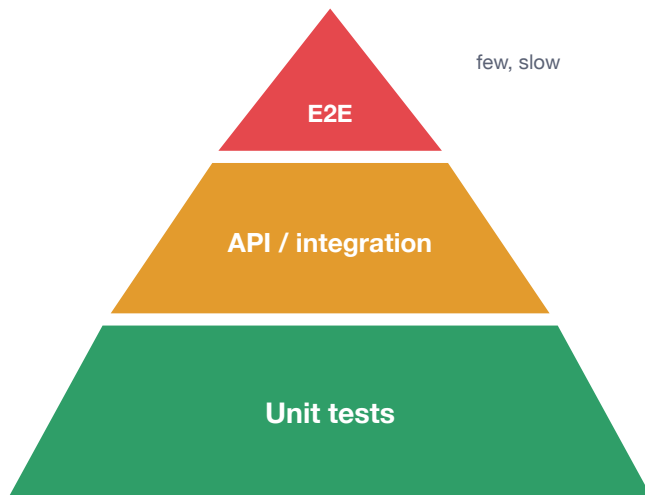


Figure 3.1 — The testing pyramid

Unit tests

Check one function in isolation. Fast and many.

API / integration tests

Send real requests to a running service and check the responses. **This whole guide lives here.**

End-to-end tests

Drive the real product the way a user would, e.g. with a browser. Few, slow, valuable.

The anatomy of a test

```
def test_buy_two_tickets():
    r = client.post("/orders", json={"ticket_id": 1, "quantity": 2}, headers=alice) # 1. act
    assert r.status_code == 201 # 2. expected answer
    assert r.json()["total_charged"] == 4000
```

The expected answer is the test. A wrong expectation is worse than no test: it goes green and certifies the bug. That's why Part 3's prompts insist on "correct" behaviour.

Happy path vs. everything else

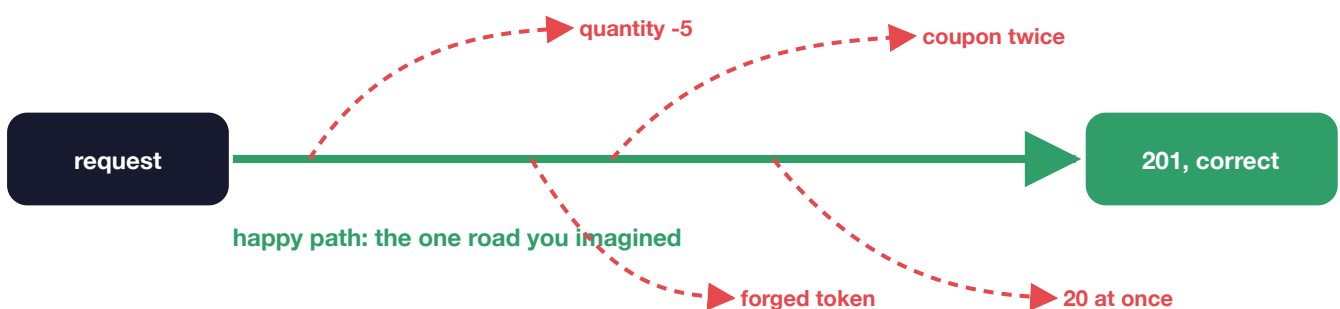


Figure 3.2 — One happy path, many unhappy ones. Each red road is a test nobody wrote.

Developers test the road they built. Attackers, confused users and concurrent traffic take the others.

Adversarial testing means walking the red roads on purpose.

Can AI find the bugs I forgot to test?

CHAPTER 4

Why bugs slip through

Not because developers are careless, but because the person who writes the code shares all of its assumptions.

When you write `quantity: int`, your brain fills in "a positive number, obviously". You then write a test with `quantity: 2`, and it passes. The assumption was never written down, so it was never tested. This is sometimes called the **curse of knowledge**: the author cannot un-know what the code is "for".

Six families of bugs this guide hunts

① Input validation

Wrong values of the right type: `-5`, `0`, empty strings, huge strings.

② Boundaries

Off-by-one at the documented limit: "up to 10" rejecting exactly 10.

③ Authentication

Who are you? Missing, malformed, forged or expired tokens.

④ Authorisation

Are you allowed? Reading another user's order by changing an id.

⑤ Business logic

Every request is valid; the combination breaks the rules. Replays, wrong formulas.

⑥ Concurrency

Two requests at the same instant both read "5 seats left".

Which ones are hard to find?

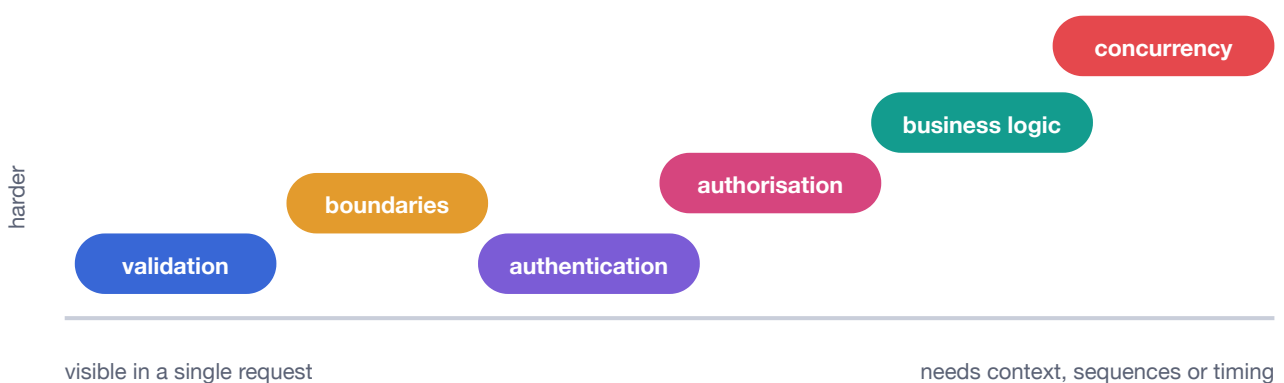


Figure 4.1 — The further right, the more a tester needs to know about the system, or the more requests it takes.

REMEMBER THIS CHART

It predicts Part 3 almost exactly. A language model reading an endpoint list is good at the left side (patterns it has seen thousands of times) and weak on the right side (rules and timing that aren't in the text it was given).

CHAPTER 5

Testing techniques compared

AI adversarial testing is not new magic. It sits in a family of techniques that already exist, and borrows from each.

Technique	How it generates inputs	Strength	Weakness
Example-based tests	A human writes each case	Precise, meaningful	Only what you imagined
Fuzzing	Random or mutated bytes, millions of times	Finds crashes nobody predicted	No idea what "wrong" means besides crashes
Property-based (Hypothesis)	Random values that satisfy a stated rule	Tests rules, not examples	You must write the rules
Schema fuzzing (Schemathesis)	Generated from the OpenAPI schema	Automatic, broad	Only knows what the schema says
Pentesting	A security expert's judgement	Deep, creative, contextual	Expensive, occasional
AI adversarial	An LLM reads docs and proposes attacks with reasons	Semantic guesses in seconds, with explanations	Can be wrong, confidently; misses what isn't in context



AI adversarial testing is a cheap, fast point between schema fuzzing and a human expert.

Figure 5.1 — A spectrum from volume to meaning.

SO IS IT "JUST FUZZING"?

No. A fuzzer sends `quantity: 48213` because the dice said so. A model sends `quantity: 10` because it read "maximum 10 per order" and suspects an off-by-one, and **tells you why**. That reason is what makes a result triageable.

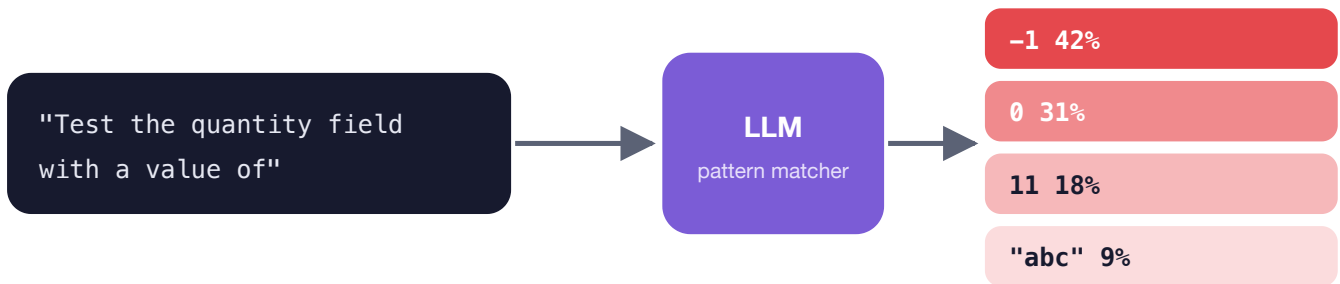
...BUT BORROW FROM FUZZING

The best setup uses both: fuzzers for volume and crashes, AI for meaning-driven hypotheses, humans for rules. They find different bugs. None replaces the others.

CHAPTER 6

What an LLM is, simply

A large language model predicts the next piece of text, over and over, using patterns learned from a huge amount of writing.



Illustrative numbers. The model picks likely continuations, then repeats for the next piece of text.

Figure 6.1 — Next-token prediction. "Testing -1 for a quantity" is extremely common in its training data, so it's a likely guess.

WHAT IT'S GOOD AT

- Recalling **common attack patterns**: SQL injection, IDOR, negative numbers, missing auth.
- Reading prose like "maximum 10 per order" and turning it into a **boundary test**.
- Producing **structured output** (JSON) quickly.
- Explaining **why** each case might matter.

WHAT IT'S BAD AT

- Knowing **rules that aren't in the prompt** (your pricing policy).
- Being **right about what "correct" means**; it can confidently expect a 500.
- **Timing and sequences** across many requests.
- Being **consistent**: the same prompt can give different answers.

Hallucination, in testing terms

A **hallucination** is fluent output that isn't grounded in reality. In this project the local model invented an endpoint called `/buy_ticket` that doesn't exist, and in another run it treated an unlimited coupon as single-use. In most AI products hallucinations are a quiet danger. In testing they are unusually **cheap**, because every guess is executed: an invented endpoint returns `404` immediately.

7B

parameters in the local model used here: small enough for a 16 GB laptop, no internet or paid API.

0.2

temperature used: low randomness, so runs are more repeatable (not perfectly).

~90s

for one generation on an Apple Silicon MacBook Air. Too slow to run live on stage.

CHAPTER 7

The big idea: AI as an adversary

Most people ask AI to write code. This talk asks it to break code, then refuses to believe it until the evidence arrives.

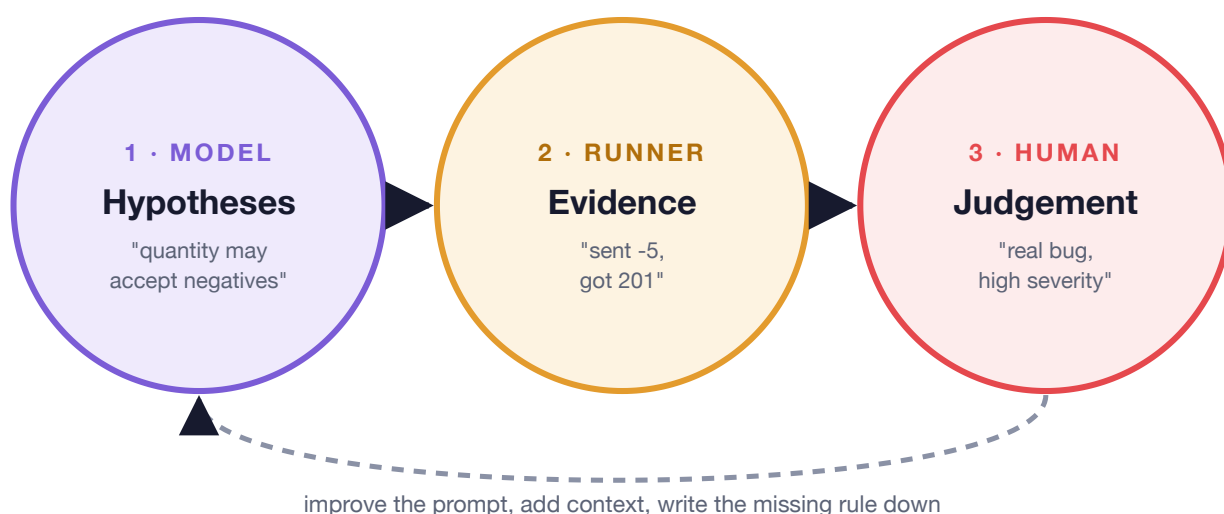


Figure 7.1 — Three roles, three different failure modes. The loop is how the process gets better over time.

Role	Its job	How it fails	Your protection
Model	Propose many plausible attacks	Invents endpoints, wrong expectations, repeats itself	Execute everything; never trust prose
Runner	Send requests, compare status codes	Can only test what its format can express	Know its limits (e.g. no concurrency)
Human	Decide what is actually a defect	Skipping triage; believing green lines	Triage every red; distrust single greens

WHY THE TALK TITLE IS A QUESTION

"Can AI find the bugs I forgot to test?" The honest answer, shown with evidence in Part 3: **some of them, quickly, mixed with confident mistakes, and not the expensive ones that depend on your business rules.** That answer is more useful than "yes" or "no", and it is what the audience should leave with.

CHAPTER 8

Project overview

A tiny, realistic, deliberately broken ticket shop, plus the tooling to let an AI attack it and a runner to check what the AI claims.

Small (about 300 lines, every one explainable on stage), **realistic** (login, search, orders, money) and **deterministic** (a reset endpoint rebuilds the database so every run starts identical).

Repository map

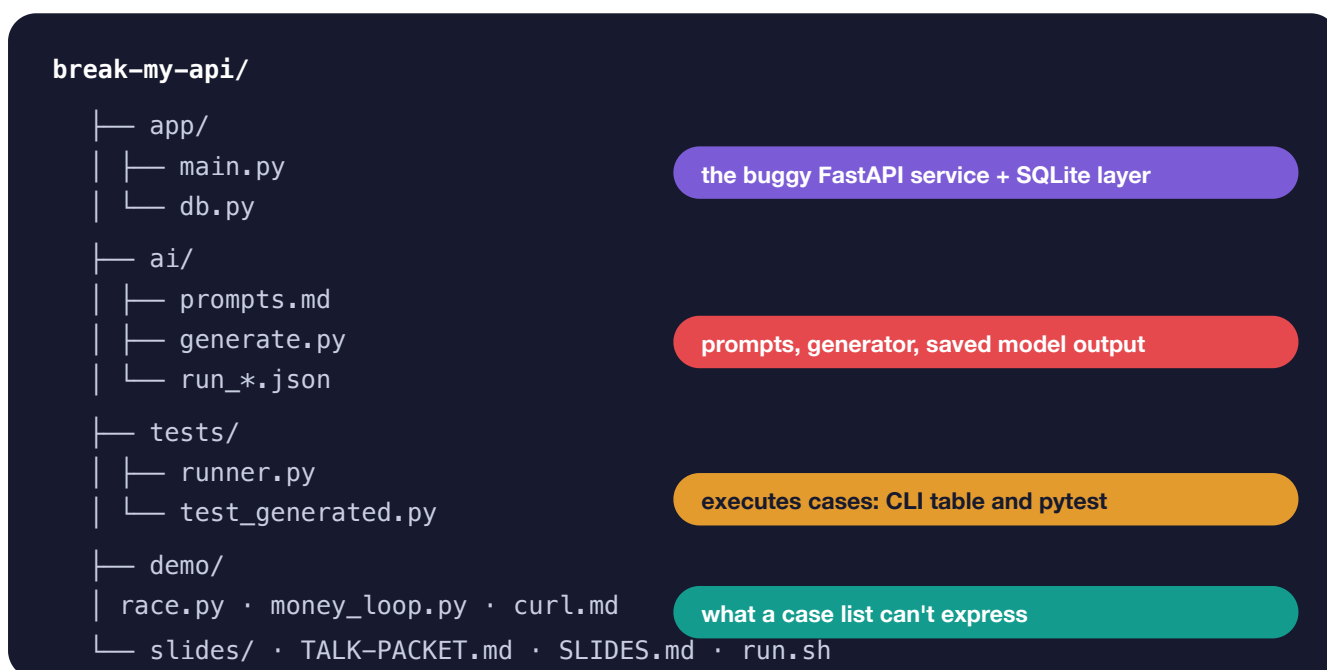


Figure 8.1 — Four folders, four jobs: the target, the attacker, the judge, and the blind spots.

Tech stack, and why

Piece	Choice	Why this and not something bigger
Web framework	FastAPI + Uvicorn	Readable, auto-generates an OpenAPI spec the AI can read.
Validation	Pydantic	Checks types automatically, which makes the <i>missing</i> value checks stand out.
Database	SQLite (one file)	No server, resets in milliseconds.
HTTP client / tests	httpx + pytest	Same cases run as a coloured CLI table or as a CI suite.
Model	Ollama + qwen2.5-coder:7b	Runs offline on a laptop; no paid API, no internet on stage.

CHAPTER 9

Architecture

Everything runs on one laptop. Nothing talks to the internet.

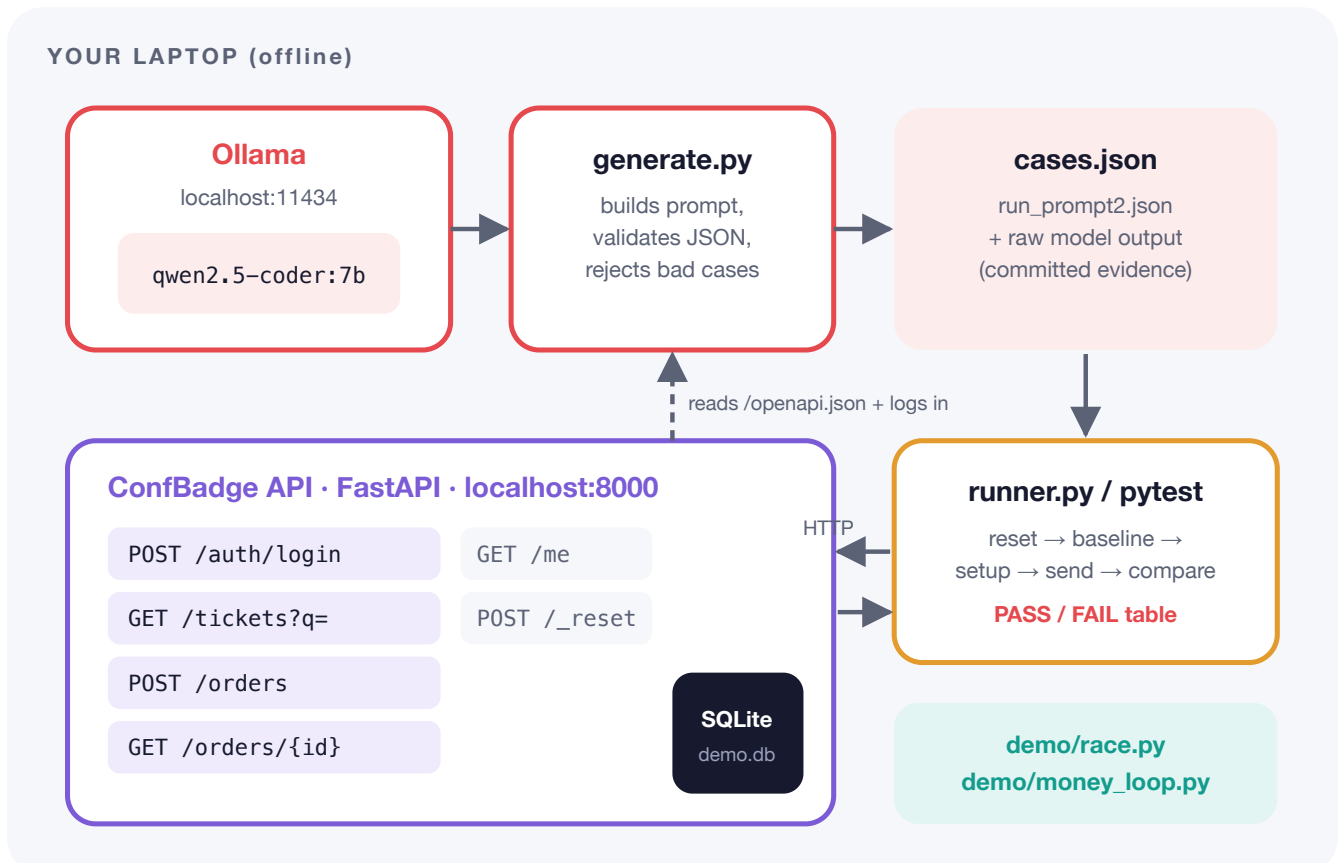


Figure 9.1 — Components and data flow. Solid arrows are the main pipeline; the dashed one is the generator reading the live API.

THE PIPELINE IN ONE LINE

generate.py asks the model for cases → saves JSON → **runner.py** sends each case to the API → compares the status with what the model said a correct API returns → prints PASS/FAIL.

WHY GENERATION IS SEPARATE

The runner never calls the model. If Ollama is slow or down on stage, you run the saved JSON and nothing downstream notices. This separation is the backbone of the backup plan.

CHAPTER 10

The ConfBadge domain

An internal conference ticket shop. Attendees hold prepaid credits, buy tickets with them, and earn some credits back.

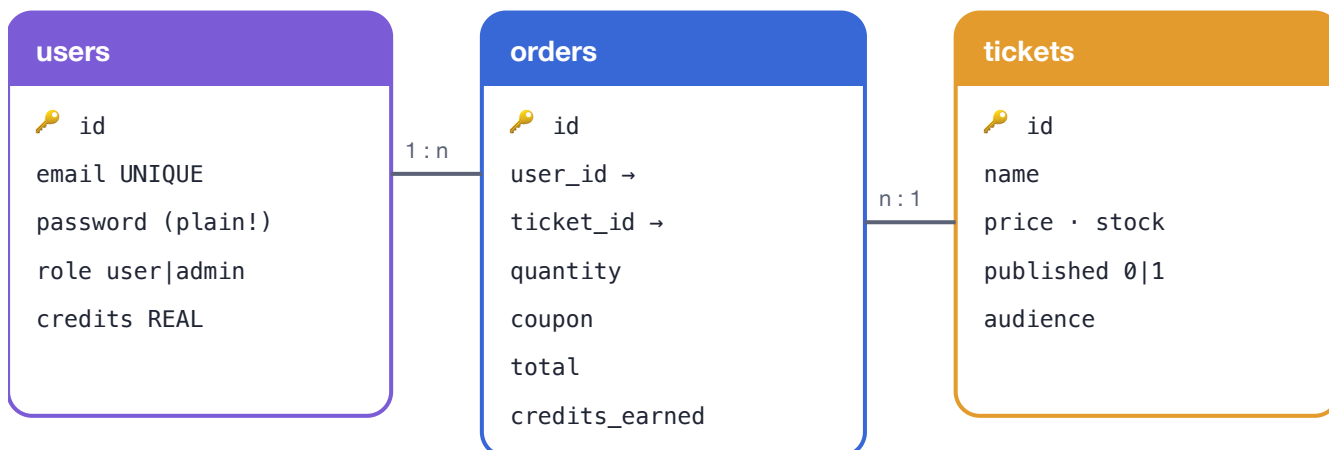


Figure 10.1 — Three tables. Note what's missing: nothing records which coupons have been redeemed.

Seed data

Ticket	Price	Stock	Visible
GENERAL-2026	2000	50	yes
STUDENT-2026	500	20	yes
WORKSHOP-AI	3500	5	yes
STAFF-COMP	0	10	hidden

User	Password	Credits
alice@corp.example	alice123	5000
bob@corp.example	bob123	5000
root@...internal (admin, id 9)	r00t	0

The business rules

1. Everyone holds a prepaid **credit balance**.
2. Buying a ticket **deducts** the price from it.
3. Every purchase returns **10% as loyalty credits**.
4. **Credits are spendable. Credits are money.**
5. `EARLYBIRD` = 20% off, unlimited. `SPEAKER100` = 100% off, **single use**.
6. Maximum **10 tickets per order**.
7. Unpublished tickets are internal only.
8. Unwritten: student pricing needs a verified student address. *This rule exists only in a "policy document", not in the code.*

CHAPTER 11

Endpoint reference

Four public endpoints, two helpers. This is the "contract" every attack is measured against.

Endpoint	Auth	Input	Success
POST /auth/login	none	{"email", "password"}	200 {access_token, role}
GET /tickets	none	?q= search text	200 {count, results[]}
POST /orders	Bearer	{ticket_id, quantity, coupon?}	201 {order_id, total_charged, credits_earned, credit_balance}
GET /orders/{id}	Bearer	order id in path	200 the order row
GET /me	Bearer	-	200 {id, email, role, credits}
POST /_reset	none (!)	-	200, database rebuilt

The happy path, end to end

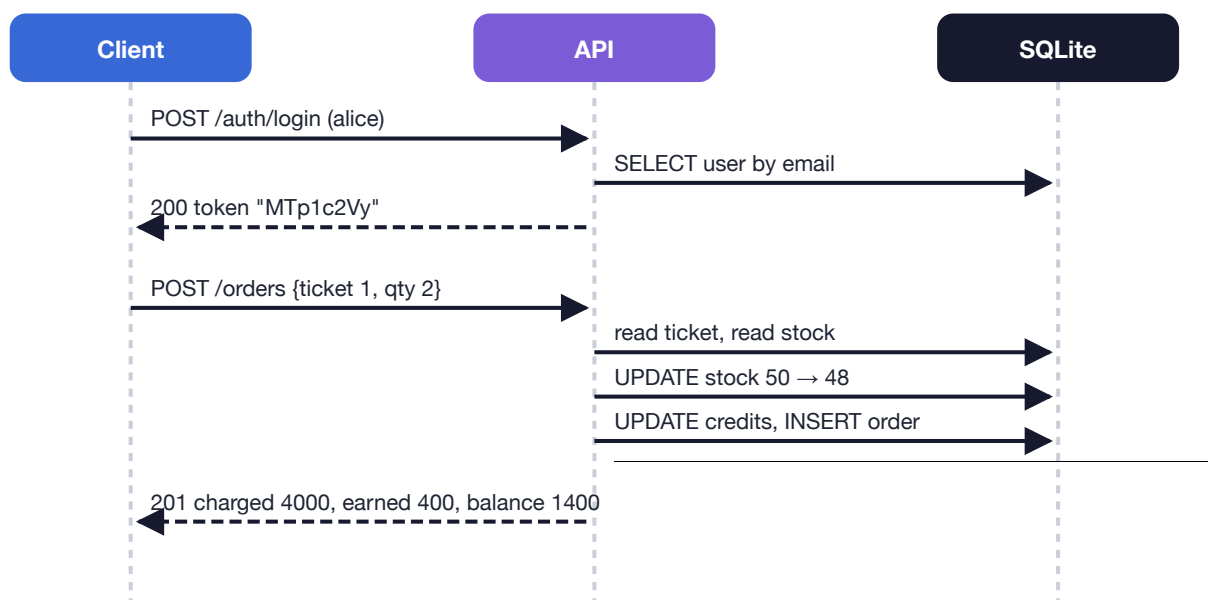


Figure 11.1 — Sequence diagram of a normal purchase. Everything here works. That is the trap.

HELPERS ARE ATTACK SURFACE TOO

`/_reset` is a test hook with no authentication, and it appears in the OpenAPI spec. A model reading the spec may call it or flag it. On stage, either hide it from the spec or call it out as one more bug.

CHAPTER 12

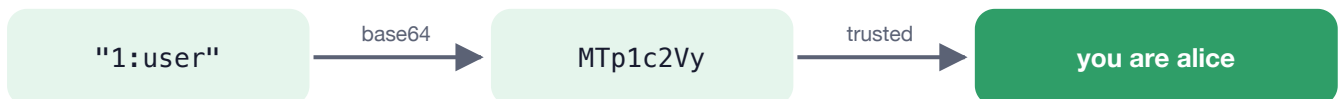
Code tour: authentication

The token looks like a security feature. It is a label anyone can print.

```
def make_token(user_id: int, role: str) -> str:
    return base64.urlsafe_b64encode(f"{user_id}:{role}".encode()).decode() # BUG-B7

def current_user(authorization):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "missing bearer token")
    try:
        raw = base64.urlsafe_b64decode(authorization[7:] + "===").decode()
        user_id, role = raw.split(":")
    except Exception as exc:
        raise HTTPException(500, f"token decode failed: {exc}") # BUG-B9
    return {"id": int(user_id), "role": role}
```

WHAT THE SERVER DOES



WHAT AN ATTACKER DOES

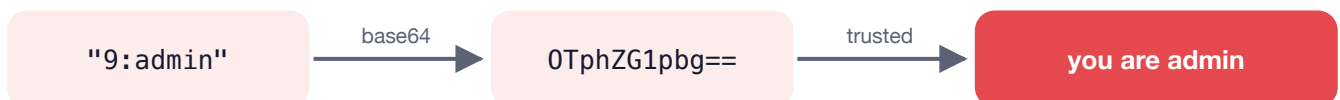


Figure 12.1 — Encoding is not signing. Nothing stops anyone from producing a token the server never issued.

WHAT'S WRONG

- No secret, no signature: anyone can mint a token.
- No expiry, no revocation.
- A malformed token returns **500** and leaks the Python exception text.
- Passwords are stored in plain text.

WHAT CORRECT LOOKS LIKE

- Sign tokens with a server secret (e.g. JWT with HS256 via PyJWT) and include `exp`.
- Any decode or signature failure returns **401** with a generic message.
- Hash passwords (bcrypt/argon2).

Code tour: placing an order

`create_order` is where money moves, so it's where most of the interesting bugs live. Red markers show where a check is missing or wrong.

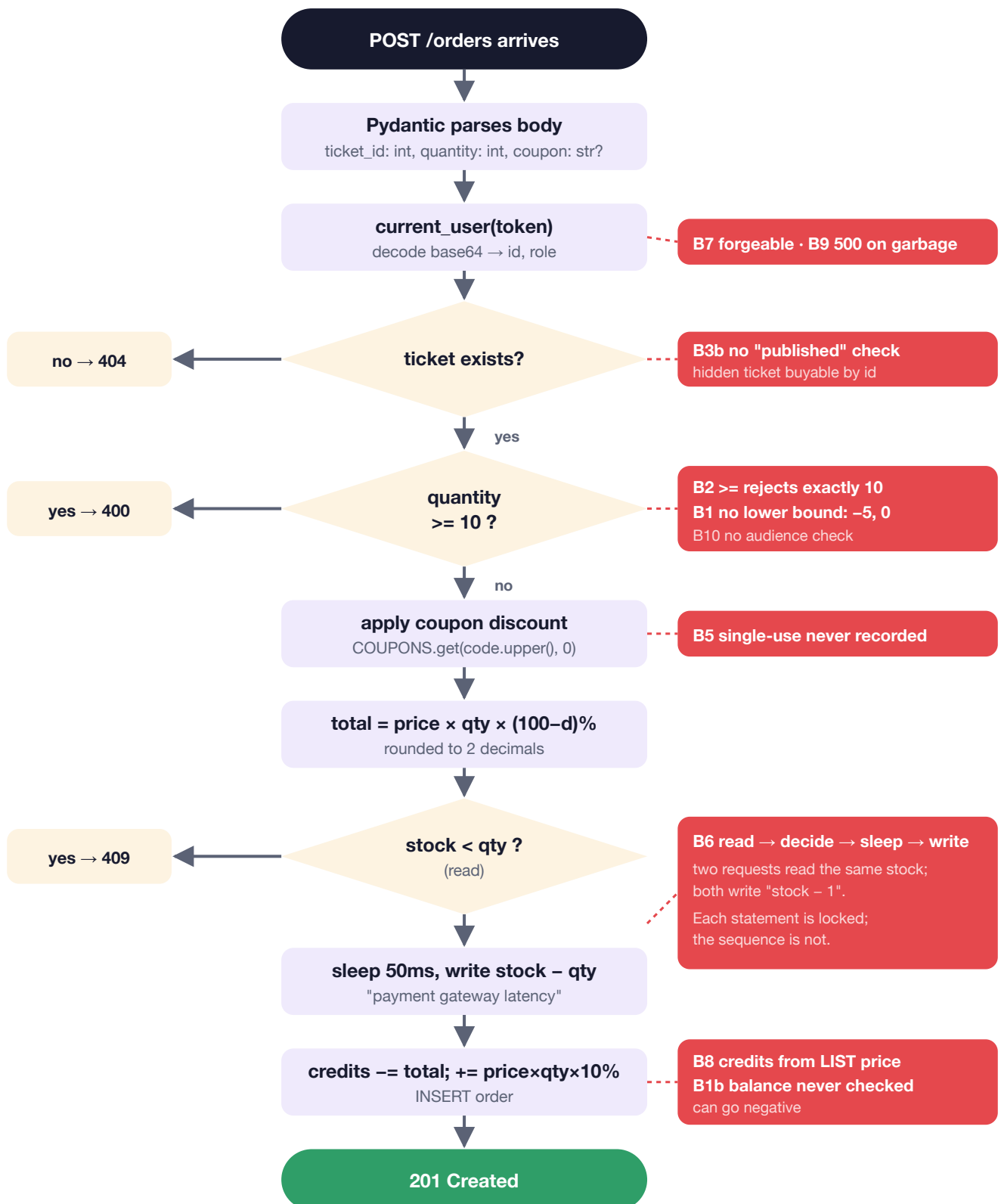


Figure 12.2 — The order flow with every intentional defect marked. Nine of the ten bugs pass through this one function.

Code tour: search and the database

Search builds SQL with string formatting

```
@app.get("/tickets")
def search_tickets(q: str = ""):
    sql = ("SELECT id, name, price, stock, audience FROM tickets "
          f"WHERE published = 1 AND name LIKE '{q}%'") # BUG-B3
    rows = query_all(sql)
```

```
q = WORK
WHERE published = 1 AND name LIKE '%WORK%'
```

```
q = '% OR published = 0 OR name LIKE '%
WHERE published = 1 AND name LIKE '%%' OR published = 0 OR name LIKE '%%'
The attacker's quote closes the string; the rest becomes SQL. The hidden STAFF-COMP ticket is returned.
```

Figure 12.3 — How input becomes code.

A lock that looks thread-safe

```
DB_LOCK = threading.Lock()

def query_one(sql, params=()):
    with DB_LOCK:
        # each statement is protected...
        return _conn.execute(sql, params).fetchone()

def execute(sql, params=()):
    with DB_LOCK:
        # ...but read → decide → write is three separate
        # calls
        cur = _conn.execute(sql, params); _conn.commit(); return cur.lastrowid
```

THE LESSON

Locking each statement feels safe but protects nothing that matters. Correctness needs the **whole decision** in one atomic statement or transaction. This is the most realistic bug in the project; it's also the one a "one request, one expected status" test format cannot catch.

Reset: why every demo is identical

```
def init_db():
    with DB_LOCK:
        close old connection
        delete demo.db
        create schema
        insert seed data
```

`init_db()` runs on startup and on `POST /_reset`. The runner calls reset **before every single case**, then creates one baseline order for Alice, so a case gives the same verdict alone, in any order, or in the full suite.

Without this, a test like "Bob reads order 1" passes by accident whenever order 1 doesn't exist yet.

CHAPTER 13

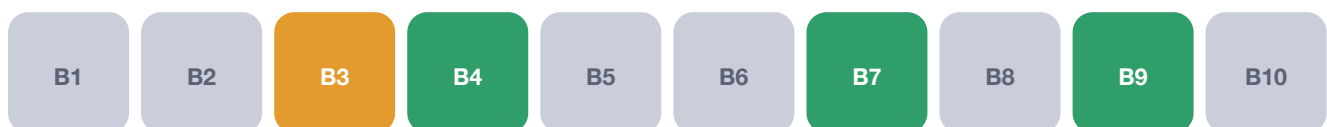
The ten intentional bugs: map

Every bug is marked `# BUG-Bn` in `app/main.py`. The next ten pages take them one at a time.

ID	Bug	Family	Severity	Saved case set	Local 7B run
B1	Negative/zero quantity, balance unchecked	validation	high	exposed	not tried
B2	"Up to 10" rejects exactly 10	boundary	low	exposed	tried 11 only
B3	SQL injection in search; hidden ticket buyable	injection	critical	exposed	wrong oracle
B4	Any user reads any order (IDOR)	authorisation	high	exposed	found
B5	Single-use coupon reusable	business logic	high	exposed	not tried
B6	Oversell race on stock	concurrency	high	no	no
B7	Forgeable bearer token	authentication	critical	exposed	found
B8	Credits from list price, not paid	business logic	critical*	no	no
B9	Malformed token → 500 + internals	error handling	medium	exposed	found
B10	Student tickets sold to anyone	unwritten rule	medium	no	no

* B8 alone is harmless-looking; chained with B5 it creates money without limit. "Saved case set" is `ai/generated_cases.json`, whose origin is not a verified run of this model. "Local 7B run" is the real `qwen2.5-coder:7b` output from 17 Sep 2026.

Real local-model run vs. all 10 bugs



■ found with a correct expectation (3)
 ■ touched, but wrong expectation (1)
 ■ not found (6)

Plus two false alarms that are not bugs at all (EARLYBIRD "reuse", and expecting a 500 for injection). See chapter 19.

B1 Negative quantity, unchecked balance

input validation

severity: high

LLM difficulty: easy

local 7B run: not tried

The bug

```
class OrderIn(BaseModel):
    quantity: int          # any int: -5, 0, 999999
    ...
    if body.quantity >= MAX_PER_ORDER: raise HTTPException(400)
    # no lower bound, and the balance is never checked before charging (B1b)
```

WHY HAPPY-PATH TESTS MISS IT

Nobody writes a test for buying minus five of something. Pydantic makes the field *feel* validated because it rejects "abc", but type checks aren't range checks.

THE ATTACK

```
POST /orders (as alice)
{"ticket_id": 1, "quantity": -5}
```

What the vulnerable API returns

```
201 Created
{"ticket": "GENERAL-2026", "quantity": -5,
 "total_charged": -10000.0, "credits_earned": -1000.0, "credit_balance": ...}
```

balance before

B

$$-(-10000) + (-1000)$$

balance after

B + 9000

Figure B1 — A "purchase" that pays the buyer 9,000 credits and adds 5 seats to stock.

The fix

```
from pydantic import Field
quantity: int = Field(ge=1, le=10)
...
if user_credits < total:
    raise HTTPException(402, "insufficient credits")
```

Demo it live

One curl from [demo/curl.md](#), or point at the red TC-001 row. Say: "Two-oh-one. For minus five tickets. And my balance went up."

Variant: order 5 workshop tickets (17,500) with 5,000 credits: accepted, balance goes negative.

B2 Off-by-one at the documented limit

boundary value

severity: low

LLM difficulty: easy, if the docs are in context

local 7B run: tried 11, never 10

The bug

```
MAX_PER_ORDER = 10          # docs: "Maximum 10 tickets per order"
if body.quantity >= MAX_PER_ORDER:    # rejects 10 itself
    raise HTTPException(400, f"max {MAX_PER_ORDER} tickets per order")
```

Boundary testing: always test the limit, one below, and one above

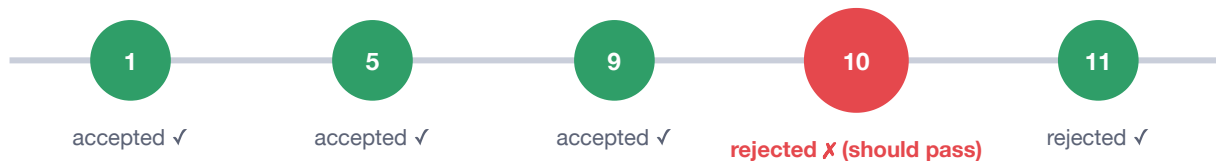


Figure B2 — The documented maximum is the one value that behaves wrongly.

WHY HAPPY-PATH TESTS MISS IT

Tests use comfortable numbers like 2. Even a "limit test" often checks 11 is rejected, which passes, and stops there. The local model did exactly this: four near-identical cases, all with 11.

THE ATTACK

`{"ticket_id": 1, "quantity": 10}` with expectation **201**. Response: **400** max 10 tickets per order.

The fix

```
if body.quantity > MAX_PER_ORDER:
```

Or better, let the schema carry it: `Field(ge=1, le=10)`. Then the OpenAPI spec documents the limit too, and schema-driven tools can test it automatically.

Is it really a bug?

Arguable, and that is a good talking point. The code and the docs disagree. **Someone has to decide which one is right.** The model can't; it rated this "medium" with no idea whether customers ever order 10.

B3 SQL injection and hidden tickets

injection · access control

severity: critical

LLM difficulty: easy to suggest, hard to get the right payload and oracle

The bug (two parts)

```
# B3: search concatenates user input into SQL
f"WHERE published = 1 AND name LIKE '{q}%'
# B3b: ordering never checks "published", so a leaked id can be bought
ticket = query_one("SELECT * FROM tickets WHERE id = ?", (body.ticket_id,))
```

Payload for <code>q</code>	Status	What really happened
' OR '1'='1	200	Published tickets only. Looks safe. It isn't.
%' OR published = 0 OR name LIKE '%	200	All 4 tickets, including hidden STAFF-COMP .
' (single quote)	500	SQLite syntax error: a strong injection signal.

THE GREEN THAT LIED

The classic payload ' OR '1'='1 doesn't leak anything here because of how the `LIKE` clause is shaped. A test using only that payload goes **green on a vulnerable API**. One passing test proves that one guess failed, nothing more.

WHAT THE LOCAL MODEL DID

It sent an injection payload but wrote `expect_status: [500]` as the *correct* behaviour. The API returned 200, so the case went red for the wrong reason: a **false alarm with the wrong oracle**, sitting right next to a real bug it didn't prove.

The fix

```
rows = query_all(
    "SELECT ... WHERE published = 1 AND name LIKE ?",
    (f"%{q}%",))
# and in create_order:
if ticket is None or not
ticket["published"]:
    raise HTTPException(404)
```

Demo it live: run TC-006 (green) and TC-007 (red) side by side. Same idea, different payload, opposite verdicts. Then order `ticket_id: 4` directly: **201** for a staff-only ticket.

B4 Reading other people's orders (IDOR)

authorisation

severity: high

LLM difficulty: easy to suggest

local 7B run: found (TC-005)

The bug

```
@app.get("/orders/{order_id}")
def get_order(order_id: int, authorization=Header(None)):
    current_user(authorization)          # authenticated...
    row = query_one("SELECT * FROM orders WHERE id = ?", (order_id,))
    return dict(row)                     # ...but never authorised
```

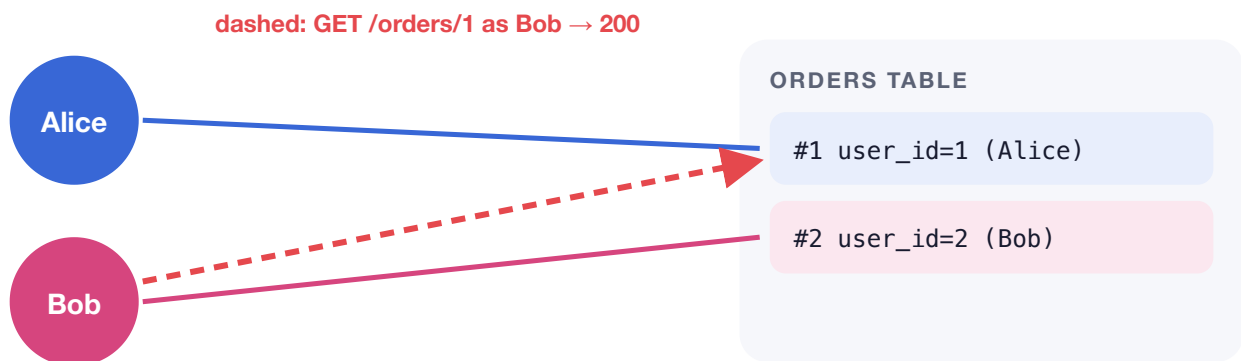


Figure B4 — Insecure Direct Object Reference: change a number in the URL, read someone else's data.

WHY HAPPY-PATH TESTS MISS IT

Test suites usually have **one** test user. With one user, an ownership bug is invisible by construction. The runner has two (Alice and Bob) precisely for this.

THE FIX

```
if row is None or row["user_id"] != user["id"]:
    raise HTTPException(404)
```

404, not 403: a 403 confirms the order exists.

Real run note: the local model found this with `expect_status: [403]`. The API returned 200, so it's a real finding. A human might still correct the expectation to 404; that's triage, not a failure.

B5 The single-use coupon that never runs out

business logic · replay

severity: high

LLM difficulty: medium

local 7B run: tested the wrong coupon

The bug

```
COUPONS = {"EARLYBIRD": 20, "SPEAKER100": 100} # SPEAKER100 documented as SINGLE USE
if body.coupon:
    discount = COUPONS.get(body.coupon.upper(), 0) # nothing ever records a redemption
```

Request 1

coupon: SPEAKER100

201 paid 0.00

Request 2

coupon: SPEAKER100

201 paid 0.00

Request N...

coupon: speaker100

201 paid 0.00

Each request is valid on its own. The defect only exists across requests.

Figure B5 — Replay: sending the same request again is the whole attack.

WHY HAPPY-PATH TESTS MISS IT

A coupon test checks that SPEAKER100 gives 100% off, once. It passes. The **"single use" rule is state**, and testing state needs a second request.

WHAT THE LOCAL MODEL DID INSTEAD

It wrote "state that must only be used once" and then sent `EARLYBIRD`, which is documented as **unlimited**, once, expecting 400. The API correctly returned 201. A confident false alarm.

The fix

```
CREATE TABLE redemptions (
  coupon TEXT PRIMARY KEY, user_id
  INTEGER);
# inside the same transaction as the order:
INSERT INTO redemptions VALUES (?, ?)
# unique-constraint error → 409 "coupon
already used"
```

Demo it live

TC-012 in the saved case set sets `"repeat": 2`. Point at the red row: two 201s, both free. Then say *"hold on to this one"*. It's half of the money loop in B8.

B6 Twenty buyers, five seats

concurrency · race condition

severity: high

LLM difficulty: may suggest it; can't execute it in this format

The bug

```
stock = query_one("SELECT stock FROM tickets WHERE id = ?", ...)[ "stock"] # 1. read
if stock < body.quantity: raise HTTPException(409, "sold out") # 2. decide
time.sleep(0.05) # 3. gap
execute("UPDATE tickets SET stock = ? WHERE id = ?", (stock - body.quantity, ...)) # 4. write
```

Timeline (ms)

Request A

read 5

sleeping (payment)

write 4

Request B

read 5

sleeping (payment)

write 4

...18 more

read 5

sleeping

write 4

0

20

50

70

Figure B6 — Everyone reads "5" before anyone writes. All 20 orders pass the check.

20

orders accepted by
demo/race.py

0

rejected as sold out

2–3

seats still "remaining" (varies run to run; the oversell doesn't)

The fix: one atomic statement

```
cur = execute("UPDATE tickets SET stock =
stock - ?"
    " WHERE id = ? AND stock >= ?", (q, id,
q))
if cur.rowcount == 0: raise
HTTPException(409)
```

WHY THE AI RUNNER CAN'T CATCH IT

The case format is **one request** → **one expected status**. "Twenty at the same instant" can't be written in it. That's a limit of the harness, not of the model's intelligence.

B7 A token anyone can forge

authentication

severity: critical

LLM difficulty: easy once it sees a sample token

local 7B run: found (TC-004)

The attack in three lines

```
# we never asked the server for this token. We made it up.
FORGED=$(python3 -c "import base64;print(base64.urlsafe_b64encode(b'9:admin').decode())")
curl -s localhost:8000/me -H "authorization: Bearer $FORGED"
```

```
200 OK
{"id": 9, "email": "root@confbadge.internal", "role": "admin", "credits": 0.0}
```

BASE64 (what this API uses)

OTphZG1pbG==

reversible by anyone
no secret involved

SIGNED (JWT HS256)

header.payload.
signature

change one byte →
signature fails → 401

PLUS

exp: expires
iat: issued at
revocation list
role from DB, not token

Figure B7 — Encoding hides nothing. Signing proves the server issued it.

WHY HAPPY-PATH TESTS MISS IT

Tests log in and use the token they got back.
They never construct one. Only someone who
looks at the token itself notices it decodes to
1:user .

THE FIX

```
token = jwt.encode({"sub": uid, "exp":  
exp},  
SECRET,  
algorithm="HS256")  
# decode with jwt.decode(...,  
algorithms=["HS256"])  
# any exception → 401
```

CONTEXT MATTERS

Prompt 2 pastes a **real login response** including the token string. A model that sees MTp1c2Vy has a chance to recognise base64. Leave it out and this bug is much less likely to be proposed. What you put in the prompt shapes what gets found.

B8 The money loop

business logic · chained bugs

severity: critical (with B5)

LLM difficulty: hard

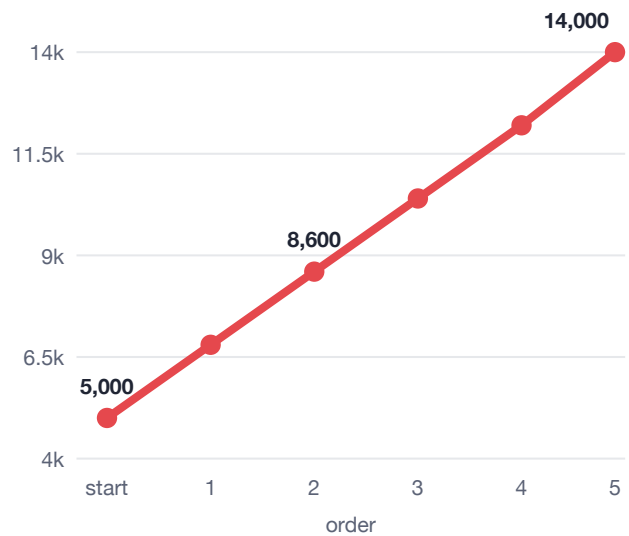
local 7B run: not found

The bug: one word in one line

```
total = round(ticket["price"] * body.quantity * (100 - discount) / 100, 2)
credits_earned = round(ticket["price"] * body.quantity * LOYALTY_RATE, 2) # LIST price, not total
```



demo/money_loop.py, real output



0.00
paid in every round

5 × 201
every request accepted

+9,000
credits from nothing, stopped only by stock

WHY EVERYONE MISSES IT

Each bug alone is boring: "coupon reusable" sounds like a few free tickets; "credits from list price" is correct for every order without a 100% coupon. **The danger only exists in the combination**, and "unbounded" is a property of a loop, not of any request.

THE FIX (EITHER KILLS THE LOOP)

```
credits_earned = round(total * LOYALTY_RATE, 2)
```

...and record coupon redemptions (B5). Fix both.

B9 A crash that talks too much

error handling · information leak

severity: medium

LLM difficulty: trivial

local 7B run: found (TC-002)

The attack and the response

```
GET /orders/1
Authorization: Bearer not-a-real-token!!

500 Internal Server Error
{"detail": "token decode failed: 'utf-8' codec can't decode byte 0x9e in position 0: invalid start byte"}
```

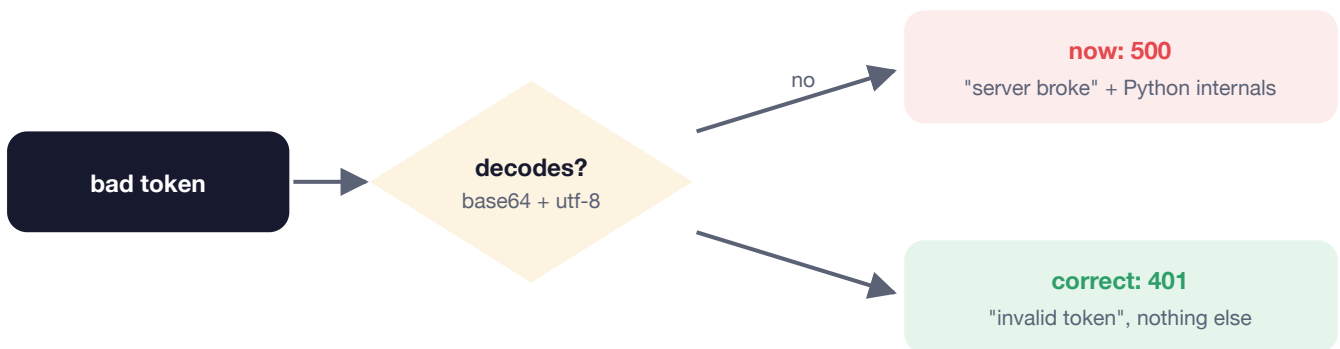


Figure B9 — The client made the mistake, so the answer should be a 4xx, and it should reveal nothing about the implementation.

WHY IT MATTERS

- Monitoring counts it as a **server error**: alerts fire for client garbage.
- Error text tells an attacker the **token format and language**.
- Tokens like `abc:user` hit `int()` outside the `try` and crash differently.

THE FIX

```
except Exception:
    raise HTTPException(401, "invalid token")
# and parse int(user_id) inside the try
```

AN HONEST GRADE

This is the kind of bug AI adversarial testing finds most reliably: common pattern, visible in one request, obvious correct answer. It's unglamorous. Saying so keeps the talk credible: **most of what this technique finds is this grade of bug**, and that is still useful.

B10

The rule that isn't in the code

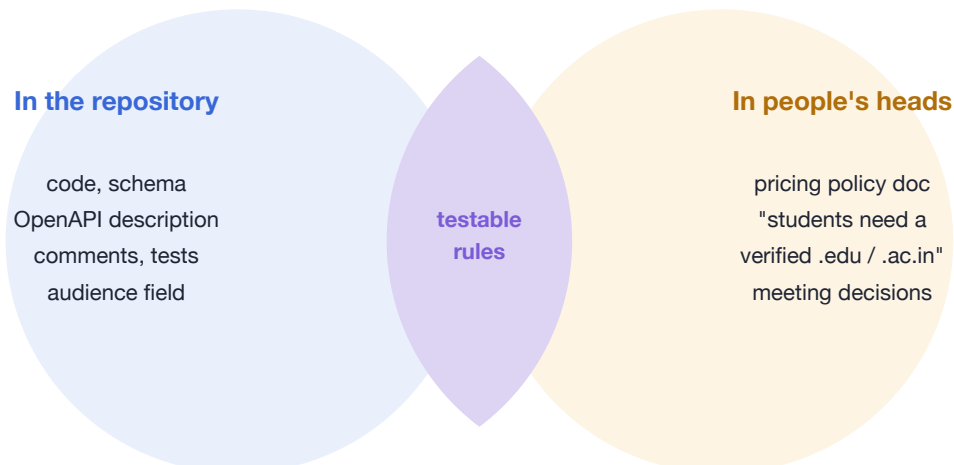
unwritten business rule

severity: depends on the policy

LLM difficulty: can guess; cannot confirm

The situation

`STUDENT-2026` costs 500 against 2000 for a general ticket. Every search result shows `"audience": "student"`. Nothing in the code checks who buys it. Alice, with a corporate email address, can buy it and gets a `201`.



A model, or a new tester, only sees the left circle. Only rules in the overlap can be checked against the code.

Figure B10 — Where business rules live, and why some can't be tested by anyone who wasn't told.

CAN AI FIND IT?

It can **guess**: a model may well try buying the student ticket as Alice. What it can't do is say whether that `201` is wrong. Neither can a human who only has the repository. **The information isn't there.**

THE FIX IS MOSTLY NOT CODE

Write the rule where readers find it: the endpoint description, the schema, a test. Then enforce it: `if ticket["audience"] == "student" and not user.verified_student: raise 403`.

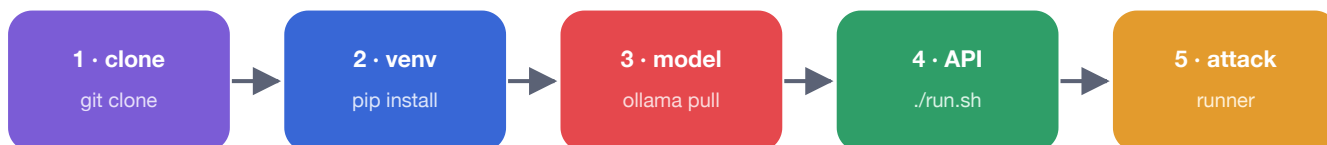
THE TAKEAWAY FOR THE AUDIENCE

It's not "AI is limited". It's **"put your business rules where a reader can find them"**, because now "a reader" includes a tool that will test them for free.

CHAPTER 14

Running the project locally

From a fresh clone to a red test table in about five minutes, plus a 5 GB model download done in advance.



```

# 1-2. project + Python environment (once)
git clone https://github.com/vidyarthna/break-my-api && cd break-my-api
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt

# 3. local model (once, ~4.7 GB; do it on good Wi-Fi, never at the venue)
brew install ollama && brew services start ollama && ollama pull qwen2.5-coder:7b

# 4. terminal A: the API
./run.sh                                     # http://127.0.0.1:8000, docs at /docs

# 5. terminal B (activate the venv here too)
python -m tests.runner                       # saved case set
python ai/generate.py --prompt 2 --out ai/run_prompt2.json
python -m tests.runner --cases ai/run_prompt2.json
python demo/race.py && python demo/money_loop.py
pytest tests/ -q                             # same cases, CI-shaped (fails on purpose)
  
```

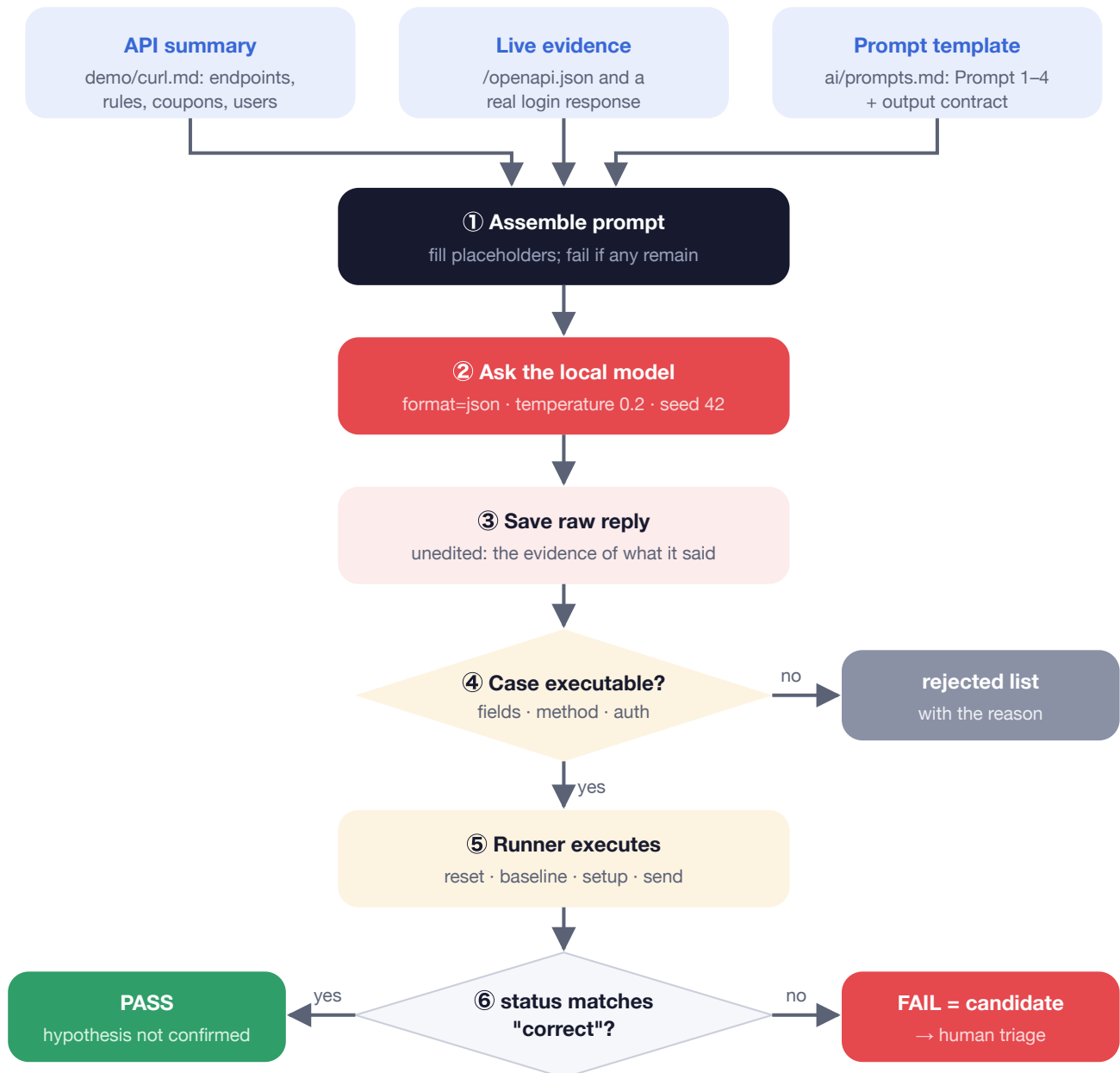
Troubleshooting

Symptom	Cause	Fix
<code>zsh: command not found: python</code>	venv not active	<code>source .venv/bin/activate</code>
<code>command not found: ollama</code>	Ollama not installed	<code>brew install ollama</code> first
cannot reach the API	API not running	start <code>./run.sh</code> in another tab
Port 8000 in use	old server still running	<code>lsof -ti:8000 xargs kill</code>
Git: "Unable to read current working directory"	terminal is in a deleted/moved folder	<code>cd ~/Downloads/break-my-api</code>
Results look different each run	model non-determinism	keep and commit the JSON you present

CHAPTER 15

The pipeline, end to end

Model output is text. The whole trick is turning that text into something a computer can execute and a human can judge.



Steps ①–④ are ai/generate.py. Steps ⑤–⑥ are tests/runner.py or pytest. They share only a JSON file.

Figure 15.1 — From documentation to a candidate-defect list.

READ THE COLOURS OF THE RESULT CAREFULLY

FAIL means "the API did not do what the model said a correct API does". That's a *candidate*, not a bug: the model's idea of correct can be wrong. **PASS** means "this guess wasn't confirmed", not "no bug here".

CHAPTER 16

Prompt design principles

Five decisions turn "write tests for my API" into a useful attacker.

1 · Ask to break, not to test

"Write tests" gets happy paths, because most test files look like that. "Assume the developer was competent but rushed and the non-obvious validation is missing" gets attacks.

2 · Demand an oracle, not a mirror

expect_status is what a **CORRECT** implementation would return, not what you predict this one returns. Without it, the model writes down current behaviour and every test passes.

3 · Structure the output

JSON with fixed fields. Prose is a blog post; a schema is a test suite. The runner only executes method, endpoint, auth, payload, expected status.

4 · Feed artefacts, not summaries

Paste the real token string. Paste the rules in plain words. The model can only reason about what's in front of it.

5 · Constrain the vocabulary

Identities are a fixed list: `none | alice | bob | garbage | forged_admin`. The runner knows how to turn each into a header. Free-text auth would be unexecutable. (The 7B model still put a raw token in this field once; the generator rejected that case.)

The output contract

```
{ "assumptions": [...],
  "cases": [{
    "method": "POST",
    "endpoint": "/orders",
    "auth": "alice",
    "payload": {...}, "repeat": 1, "setup": [],
    "expect_status": [400, 422],

    "id", "objective", "severity",
    "expected_behaviour",
    "potential_vulnerability",
    "reason": "A3: ..."
  ]
}
```

EXECUTED

These fields hit the wire.
A hallucinated endpoint here fails loudly with a 404.

READ BY HUMANS

Never executed. Used for triage; a wrong reason costs a sentence, not a false result.

Figure 16.1 — Separate what executes from what explains. This is the main defence against hallucination.

CHAPTER 17

The four prompts

Four levels, one shared output contract. Full text in `ai/prompts.md`; print any assembled prompt with `--print-prompt`.

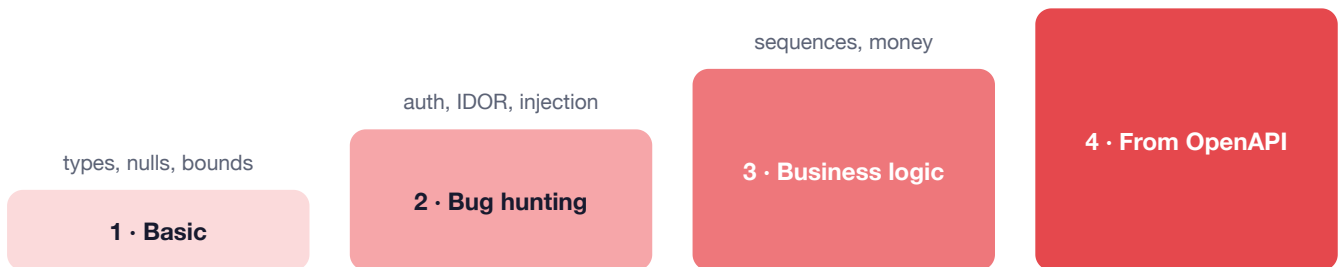


Figure 17.1 — Each level needs more context and more reasoning from the model.

Prompt 1 · Basic adversarial testing

```
You are an adversarial API tester. Your job is to make this API behave badly,
not to confirm that it works.
Here is the API: <endpoint summary>
Generate 10 test cases targeting: missing required fields · null where a value is
expected · wrong JSON types · empty strings and objects · boundary values
(0, -1, maximum, maximum+1) · oversized values.
Do not generate a single case that you expect to succeed.
<output contract>
```

Use it for: input validation and boundaries (B1, B2). **Expect:** many easy, low-severity findings, quickly.

Prompt 2 · Advanced bug hunting

```
You are a senior security engineer doing a black-box review of an internal API.
Assume the developer was competent but rushed. Assume the obvious validation is
present and the non-obvious validation is missing.
API under test: <endpoint summary> <sample login response, including raw token>
Fill the "assumptions" array with five assumptions this API makes about its
callers. Then write cases that violate each one; start each "reason" with "A3: ...".
Cover at minimum: authentication (missing, malformed, expired, forged, another
user's) · authorisation (A reads B's objects) · injection in free-text parameters ·
resources hidden from listings but reachable by id · numeric fields used in
arithmetic · state that must only be used once.
Rank by exploitability, not by how easy the case was to write.
```

Use it for: auth, IDOR, injection, replay (B3, B4, B5, B7, B9). This is the main demo prompt.

Prompts 3 and 4

Prompt 3 · Business-logic attack

```
Forget input validation. Assume every field is well-formed and every request is
authenticated. I want you to attack the ECONOMICS of this API.
API under test: <endpoint summary> # ← added after the first real run; see ch. 19
Here are the rules of the system:
- Users hold a prepaid credit balance. - Buying a ticket deducts the price.
- Buying earns loyalty credits back. - SPEAKER100 is 100% off and single use.
- Credits are spendable on future tickets.
Find sequences of individually legal requests whose combined effect is illegal.
Look for: value moving in the wrong direction · a value computed from the wrong
input · a limit enforced per-request but not per-user · an earlier step not
undone by a later one.
```

HONEST NOTE ON THIS PROMPT

"A value computed from the wrong input" nearly describes B8 outright. If a model finds B8 from this, say how much the prompt helped. In the real 7B run it found nothing economic anyway (chapter 19).

Prompt 4 · From an OpenAPI specification

```
Here is an OpenAPI 3.1 specification: <curl -s localhost:8000/openapi.json>
The specification tells you what the developer INTENDED. Your job is to find where
the intent and the implementation are likely to diverge. For each endpoint:
1. List every constraint the schema declares (types, required fields, enums).
2. List every constraint the description implies in prose but the schema does NOT
   enforce. These are the high-value targets.
3. For each item in (2), write a test case that violates it.
```

Which prompt finds which bug?

Bug	P1	P2	P3	P4	Needs beyond the prompt
B1 negative qty	•	◐		•	-
B2 off-by-one	•			•	the "10" in context
B3 injection		•		◐	a payload that fits the LIKE clause
B4 IDOR · B7 forged token		•			two users · a sample token
B5 coupon replay		◐	•		repeat field
B6 race			◐		a concurrent harness
B8 money loop			◐		rules + multi-step reasoning
B10 student rule					the policy itself

• designed to target · ◐ might surface it. This is intent, not measured success.

CHAPTER 18

Inside generate.py and runner.py

Two deliberately boring programs. Boring is what makes the evidence trustworthy.

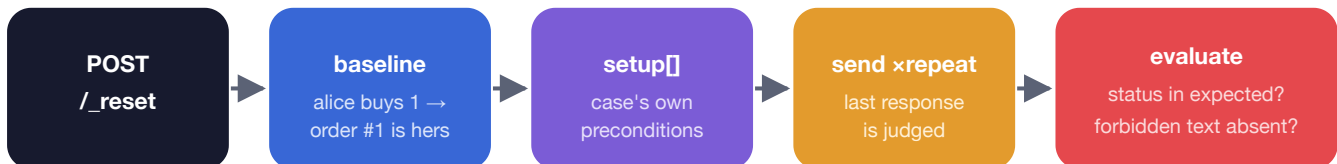
generate.py: reject, never repair

```
def problems(case):
    out = [f"missing {k}" for k in REQUIRED if k not in case]
    if method not in ("GET", "POST"): out.append("unsupported method")
    if case.get("auth") not in AUTH_VALUES: out.append("unknown auth")
    if not list_of_ints(expect_status): out.append("bad expect_status")
    if placeholder_like("<your order id>"): out.append("contains a placeholder")
    return out # non-empty → case goes to "rejected", with reasons
```

WHY NOT FIX BAD CASES AUTOMATICALLY?

An early version filled in a missing `expect_status` with `[400, 422]`. That quietly put words in the model's mouth: the "AI-generated" expectation was actually the tool's. Rejecting with a reason keeps the record honest.

runner.py: the life of one case



Every case starts from identical state, so its verdict doesn't depend on which cases ran before it.

Figure 18.1 — Isolation per case. Found necessary when "Bob reads order 1" passed when run alone, because order 1 didn't exist.

```
# CLI: coloured table
python -m tests.runner
python -m tests.runner TC-008
python -m tests.runner --cases
ai/run_prompt2.json
```

```
# CI: same cases as pytest
pytest tests/ -q
CASES=ai/run_prompt2.json pytest tests/ -q
# fails on purpose against this API
```

CHAPTER 19

Real run: Prompt 2

qwen2.5-coder:7b via Ollama on a 16 GB Apple Silicon MacBook Air, 17 Sep 2026.
Nothing below is edited.

91 s

to generate (model cold start
included)

9 → 8

cases produced → executable (1
rejected: raw token as `auth`)

5 / 8

red after execution

The model's five assumptions: A1 tokens are valid and unexpired · A2 callers have permission · A3 input has no malicious code · A4 numeric fields are in range · A5 state values are used once.

Every case, triaged

Case	What it sent	Expected	Got	Human verdict
TC-001	GET /orders/1, no token	401	401	Correct behaviour. Useful regression check.
TC-002	GET /orders/1, garbage token	401	500	Real bug B9.
TC-004	GET /orders/1, forged admin token	401	200	Real bug B7.
TC-005	GET /orders/1 as Bob	403	200	Real bug B4. (404 is the better expectation)
TC-006	/tickets?q=1' OR '1'='1	500	200	False alarm. Expected a crash as "correct"; payload leaked nothing.
TC-007	GET /orders/2 as Alice	404	404	Meaningless: order 2 doesn't exist. Not a "hidden resource" test.
TC-008	quantity 11	400	400	Correct. Missed the real boundary (10).
TC-009	EARLYBIRD once	400	201	False alarm. EARLYBIRD is documented as unlimited.

3 real bugs**2 false alarms****3 green****1 rejected**

Figure 19.1 — Of 5 red lines, 3 were real (60% precision on this run). All 3 real ones were in the "visible in a single request" families.

Real run: Prompt 3, twice

The business-logic prompt is where the talk's "wow moment" lives. On this model it produced nothing useful, and the first run exposed a mistake in the prompt itself.

Run A · prompt without endpoints · 20 s

```
POST /buy_ticket
{"ticket_type": "GENERAL-2026",
 "coupon": "SPEAKER100"}
expect 200 → got 404
```

Prompt 3 described the *rules* but never listed the *endpoints*, so the model invented one. The runner caught it instantly with a 404: a hallucination that cost nothing.

Run B · endpoints added · 72 s

```
TC-001 quantity 11 PASS
TC-002 quantity 11 + EARLYBIRD PASS
TC-003 quantity 11 + EARLYBIRD PASS
TC-004 quantity 11 + EARLYBIRD PASS
```

Four near-duplicates, all probing the per-order limit. No coupon replay, no credit arithmetic, no sequence. All green.

What Prompt 3 asked for vs. what came back

value moving the wrong way	0 cases
value from the wrong input (B8)	0 cases
per-request, not per-user limit	4 cases (qty 11)
step not undone later (replay, B5)	0 cases
multi-request sequence	0 (setup only)

And the four limit cases misread the category: "11 in one order" is a per-request limit, which the API does enforce.

Figure 19.2 — Instruction following degrades on the hardest category with a small model.

WHAT THIS SHOWS

- Business logic is the hardest family (chapter 4), and results match that.
- A 7B local model follows the **format** well and the **intent** weakly.
- The prompt itself had a bug. Execution found it.

WHAT THIS DOESN'T SHOW

- That no model can find B8. Larger models were not tested here.
- That Prompt 3 is useless; one run on one model is one data point.
- That the money loop is impossible to hypothesise from these rules.

CHAPTER 20

Triage: deciding what's a defect

The runner produces a list of disagreements. This flowchart turns each one into a decision.

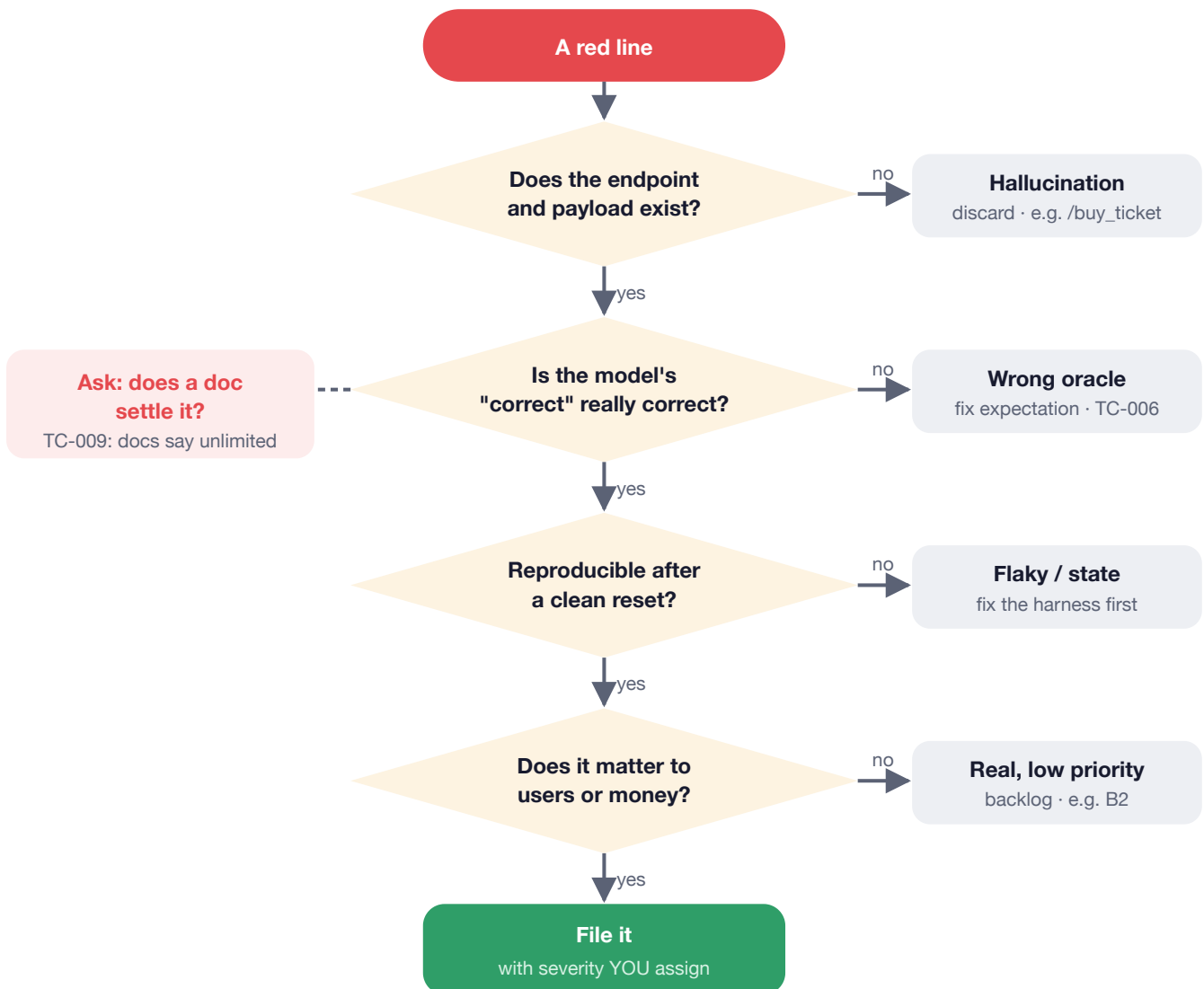


Figure 20.1 — Four questions per red line. The model can't answer any of them reliably; a person who knows the system can answer all four quickly.

AND THE GREEN LINES?

Don't skip them. Ask "did this test actually exercise the risk?" TC-007 (reading an order that doesn't exist) and the saved set's TC-006 (an injection payload that happens not to leak) are green and tell you almost nothing.

CHAPTER 21

A catalogue of AI mistakes

Every mistake below really happened in this project's runs. Each has a different cost, and a different defence.

Mistake	Real example	Caught by	Defence
Invented endpoint	<code>POST /buy_ticket</code> with <code>ticket_type</code>	runner: 404	Always include the real endpoint list.
Wrong oracle	Expected 500 as correct for SQL injection	human	Contract says "correct"; review expectations on red.
Misapplied rule	Treated unlimited EARLYBIRD as single-use	human	Put rules in plain words; check docs in triage.
Broken format	<code>"auth": "MTp1c2Vy"</code> (raw token)	generator	Fixed vocabulary + reject with reason.
Duplicates	3 identical "qty 11 + EARLYBIRD" cases	human	De-duplicate by (method, endpoint, payload).
Test that tests nothing	Reading order 2, which doesn't exist, as a "hidden resource" test	human	Distrust greens; ask what risk it exercised.
Near-miss boundary	Tested 11, never 10	nobody	Add boundary rules to the prompt or a property test.
Category confusion	"Per-user limit" tested as a per-request limit	nobody	Bigger model, or a human-written sequence test.

Cost of a mistake, by who catches it

generator

≈ 0 s: rejected before running, reason logged

runner

seconds: fails loudly (404), obvious in the table

human

minutes each: needs reading docs and code

nobody

the dangerous one: silent gaps that look like coverage

THE DESIGN GOAL

Push mistakes **up this list**. Every rule you move from "human notices" into "generator rejects" or "runner checks" makes the process cheaper and more honest.

CHAPTER 22

The honest scorecard

Two case sets, side by side. Only one of them is a verified model run, and that one is the one to present.

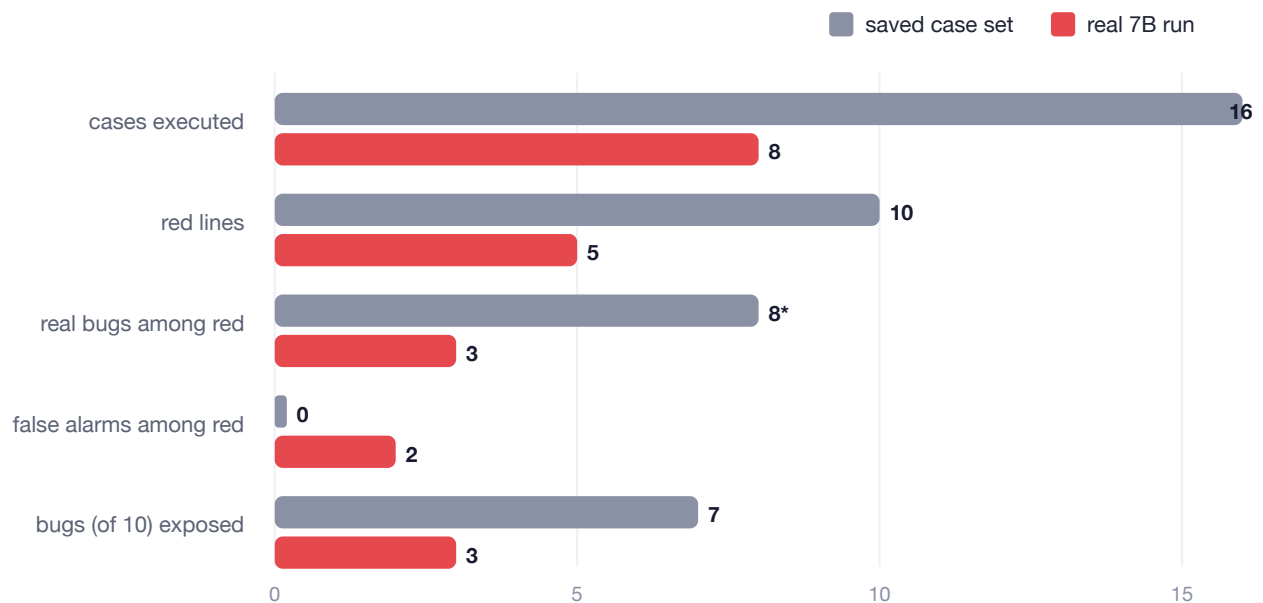


Figure 22.1 — * The saved set's 8 includes two "arguable" reds (off-by-one, zero-quantity orders): real behaviour, debatable priority.

ABOUT THE SAVED CASE SET

`ai/generated_cases.json` is cleaner than anything the local model produced, and its origin isn't a verified run. **Don't present it as this model's output.** Use it, if at all, as "a curated example of what a good run looks like".

WHY THE REAL RUN IS A BETTER TALK

3 real bugs in 90 seconds, 2 confident mistakes, nothing on the business logic, and a prompt bug found by execution. That is **exactly** "hypotheses, evidence, judgement", with no exaggeration to defend in Q&A.

Numbers to put on the scorecard slide

Metric	Value	How to say it
Generation time	91 s / 72 s	"Too slow to do live. I ran it last night; here's the unedited output."
Executable cases	8 of 9	"One came back malformed; the tool rejected it and said why."
Red / real	5 / 3	"Five red. Three were real. Two were the model being confidently wrong."
Business-logic findings	0	"The expensive bug? Nothing. That one came from a human."

CHAPTER 23

Getting better results

Each step up this ladder costs a little more and removes a specific class of misses seen in chapter 19.

more effort

6 · Human-written rules

Business rules as property tests. Nothing else finds B10.

5 · Critic pass

A second prompt reviews each case: "is this expectation really correct per the docs?"

4 · Stronger model

A larger local model or a hosted one (with data-privacy review). Likely fixes: wrong oracles, duplicates.

3 · Richer harness

Allow step sequences, state assertions (balance after), parallel groups. Fixes: B5, B6, B8 can be expressed.

2 · Several runs, merged

3–5 generations, de-duplicated. Fixes: one unlucky run; narrow coverage like "only qty 11".

cheapest

1 · Better context

Real endpoint list in every prompt; rules in plain words; sample token. Fixes: invented endpoints, misread rules.

Figure 23.1 — The improvement ladder. Start at the bottom; most teams get the biggest gain from steps 1–3.

A richer case format (sketch)

```
{ "id": "SEQ-001", "objective": "coupon replay + list-price credits create money",
  "steps": [
    { "auth": "alice", "method": "POST", "endpoint": "/orders", "payload": { "ticket_id": 1,
      "quantity": 9, "coupon": "SPEAKER100"}, "repeat": 3 }
  ],
  "assert_state": { "GET /me as alice": { "credits": { "lte": "starting_credits" } } }
}
```

Not implemented in the repo; a direction for "what next". An assertion about **final state** rather than a single status code is what lets the money loop be expressed at all.

CHAPTER 24

Using this on your own API

The demo project is a teaching toy. Here is the same pipeline pointed at a real service, one sensible step at a time.

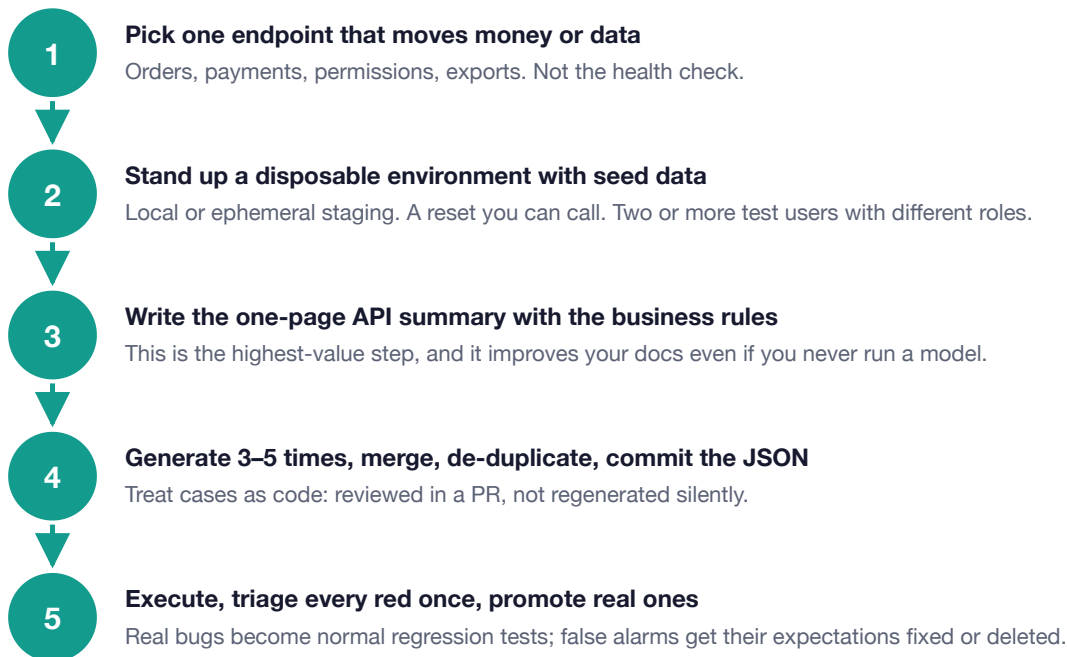


Figure 24.1 — Five steps from zero to a committed adversarial suite.

What to adapt in the code

In this repo	Hard-coded for the demo	For a real service
<code>ACCOUNTS</code> , <code>headers_for</code>	alice, bob, garbage, forged_admin	Real test identities per role; a helper that fetches tokens from your auth provider
<code>BASELINE</code> , <code>/_reset</code>	One order for Alice; an HTTP reset endpoint	Database snapshot restore, fixtures, or a fresh container per run
<code>send()</code>	GET and POST, JSON only	PUT, PATCH, DELETE, query + body, file uploads
<code>evaluate()</code>	Status code + forbidden substring	Add JSON-path assertions and state checks after the call
<code>curl.md</code> summary	Hand-written	Generated from OpenAPI + a hand-written rules section

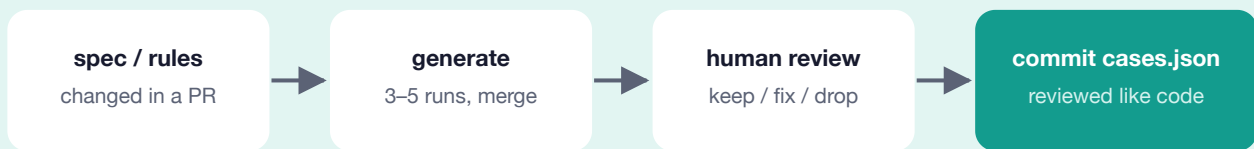
One endpoint, one week. If the first batch yields zero real findings after triage, that is useful data too: either the endpoint is solid, or your summary is missing the rules that matter.

CHAPTER 25

Putting it in CI/CD

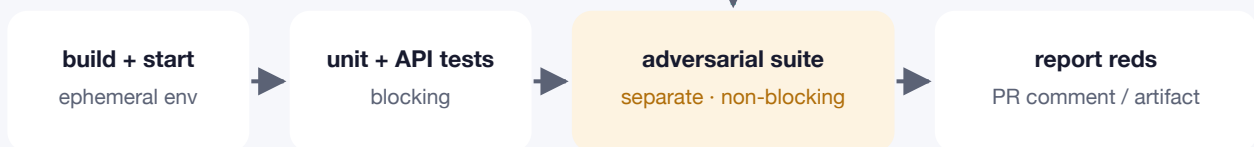
Generation is slow and non-deterministic. Execution is fast and deterministic. So they belong in different places.

OCCASIONALLY · when the spec or rules change



committed cases feed CI

EVERY PUSH · fast, deterministic



Once a red is triaged as a real bug and fixed, move that case into the blocking suite.

Figure 25.1 — Two loops. Never call a model inside the per-push pipeline.

A GitHub Actions job (sketch)

Don't regenerate cases per run, block merges on unreviewed AI cases, or point it at production. **Do** version cases, prompt and model name together, and promote confirmed findings into normal tests.

```

adversarial:
  runs-on: ubuntu-latest
  continue-on-error: true          # candidates, not build failures, until triaged
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.12" }
    - run: pip install -r requirements.txt
    - run: uvicorn app.main:app --port 8000 &
    - run: python -c "import httpx,time; [time.sleep(.5) for _ in range(20) if not
httpx.get('http://127.0.0.1:8000/docs').is_success]"
    - run: CASES=ai/cases.json pytest tests/test_generated.py -q --junitxml=adversarial.xml
    - uses: actions/upload-artifact@v4
      with: { name: adversarial-report, path: adversarial.xml }
  
```

CHAPTER 26

Security, privacy and safety

You are generating attacks and sending API details to a model. Both need guardrails.

SAFE TO SEND

endpoint names, methods
field names and types
business rules in words
synthetic seed users
tokens from a throwaway
test environment

THINK FIRST

internal hostnames
full source code
security architecture
unreleased features
→ local model, or check
your provider's data terms

NEVER SEND

production credentials
real customer data
API keys, secrets
real tokens / sessions
logs with personal data
anything under NDA

Figure 26.1 — What goes into the prompt. When in doubt, use a local model like the one in this project.

Execution safety

- **Only against environments you own** and are authorised to test. Generated payloads include injection strings.
- **Never production.** Cases create orders, change balances, reset databases.
- **Sandbox side effects:** fake payment provider, email to a sink, no real webhooks.
- **Rate-limit the runner** so a bad case can't hammer shared staging.
- **Allow-list endpoints and methods** in the runner; reject anything else (this repo accepts only GET/POST).

Local vs hosted models

	Local (Ollama)	Hosted
Data leaves laptop	no	yes
Cost	hardware only	per token
Speed here	70–90 s	usually faster
Quality	weaker (ch. 19)	usually stronger
Works offline	yes	no

PROMPT INJECTION WORKS BOTH WAYS

If your API returns user-generated text and you feed responses back into a model (for example in a "critic" step), that text can contain instructions aimed at the model. Treat API responses as data, never as instructions, and keep the executed fields under the runner's control.

CHAPTER 27

Is it worth it? Measuring value

"It found bugs" is an anecdote. Four numbers tell you whether to keep doing it.

Precision

real bugs ÷ red lines. Low precision means triage costs more than it saves. This project's real run: $3 \div 5 = 60\%$.

Novelty

findings your existing suite didn't already catch. Here: all 3, because the original tests were happy-path only.

Triage time

minutes per red line. If one red takes 20 minutes to understand, a batch of 30 is a working day.

Escapes

production bugs in categories it should have caught. The only metric that proves value over months.

A simple cost/benefit model

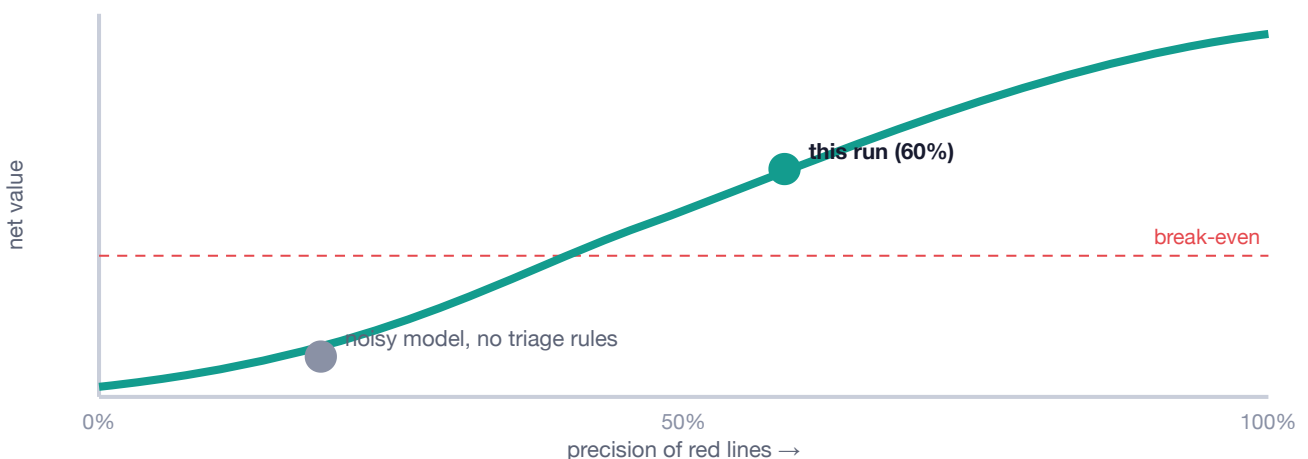


Figure 27.1 — Illustrative, not measured: value rises steeply once precision is high enough that triage stays cheap.

Keep doing it if...	Stop or change approach if...
Precision stays above roughly half	Most reds are false alarms after two prompt iterations
It finds bugs your suite didn't	Every finding duplicates an existing test
Triage averages a few minutes	Each red needs a meeting
Writing the rules summary improved your docs	No one can say what the rules are (fix that first)

CHAPTER 28

Fitting into your tools

The JSON case file is the interface. Anything that can send HTTP and compare a status can execute it.

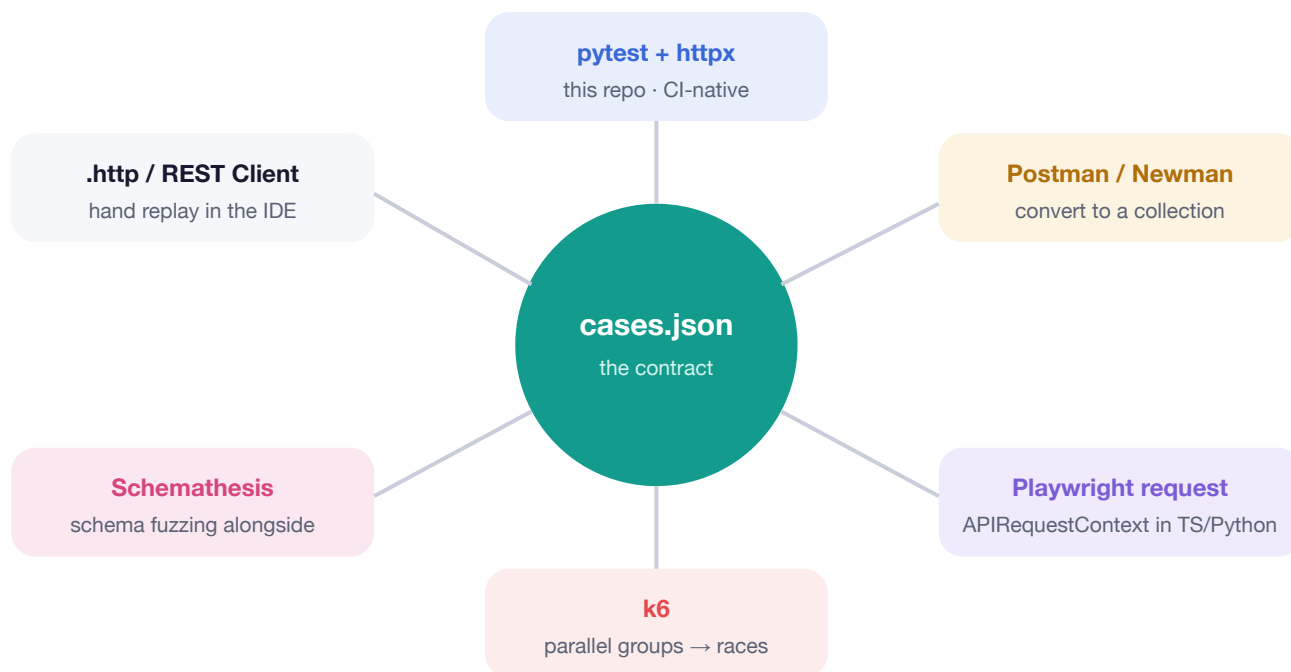


Figure 28.1 — One case format, many executors. Pick the one your team already runs.

Tool	Role alongside AI cases	Covers a gap from this guide
Schemathesis	Generates thousands of schema-valid variations	Boundaries the model skipped (B2)
Hypothesis	Property tests for business rules	"balance never increases on purchase" (B1, B8)
k6 / Locust	Concurrent traffic	Races the case format can't express (B6)
Semgrep / Bandit	Static analysis of the source	f-string SQL, catch-all 500s (B3, B9)
OWASP ZAP	Dynamic security scanning	Known vulnerability classes at volume

A PROPERTY TEST THAT WOULD CATCH THE MONEY LOOP

```

@given(qty=st.integers(1, 10), coupon=st.sampled_from([None, "EARLYBIRD", "SPEAKER100"]))
def test_buying_never_increases_balance(qty, coupon):
    before = me()["credits"]
    buy(ticket_id=1, quantity=qty, coupon=coupon)
    assert me()["credits"] <= before # fails for SPEAKER100: B8
  
```

CHAPTER 29

Tomorrow's session and its theme

HackersMang, September 2026 edition · "AI in Action: Beyond the Prompt: Engineering Real AI" · Sat 19 Sep, 10:00 IST.

Your talk is a worked example of the theme

Theme words	What they mean	Where your talk shows it
AI in Action	Real use, not slides about the future	AI-written tests run live against a real API (slides 11–12)
Beyond the Prompt	A prompt alone is not enough	The prompt gave ideas; 2 of 5 red results were still wrong (slide 14)
Engineering Real AI	The system around the model makes it trustworthy	JSON output → validation → execution → human review (slide 10)

The one idea to leave the room with

KEY SENTENCE (SLIDE 23)

AI generates hypotheses. Execution provides evidence. Humans decide what is a defect.

"The prompt is the easy part. The engineering around it is what makes AI useful."

Tie to the theme twice

AT THE START (SLIDE 1)

"Today's theme is 'beyond the prompt'. My talk is a small, real example: the prompt got AI to suggest attacks, but it was the engineering around it, running every idea and checking the evidence, that told me which ones were true."

AT THE END (SLIDE 23)

"That's what 'beyond the prompt' means to me. The prompt is the easy part. The engineering around it is what makes AI something you can trust."

What your talk is, and is not

IT IS

- A positive, practical way to use AI for testing
- Honest about real results from a real run

IT IS NOT

- "AI isn't good enough"
- A benchmark of which AI is best

CHAPTER 30

Your talk in one minute

If someone asks "what's your talk about?" in the hallway, or the host asks for a summary.

One sentence

I asked AI to break my API instead of writing it. It found real bugs fast, got some wrong, and the engineering around it is what told me which was which.

One minute (about 150 words)

"Most of us use AI to write code. I wanted to know: can it break code?

So I built a small ticket-shop API with ten bugs planted on purpose. Buy minus five tickets and you get paid. A single-use coupon works forever. A made-up token makes you admin.

Then I asked a local AI model to attack it: unexpected inputs, edge cases, the things we forget to test. It wrote eight tests last night; today we run them live.

Five came back red. Three were real bugs my own tests had missed. Two were the AI being confidently wrong.

And the most expensive bug, a coupon trick that creates money from nothing, the AI never found. I did, because I know how the business works.

So here's the lesson: AI generates guesses. Running them gives evidence. Humans decide what's actually a bug. The prompt is the easy part; the engineering around it is what makes AI useful. Let me show you."

Twenty seconds

"I asked AI to break my API. It found three real bugs my tests missed, got two wrong, and missed the one that creates money. AI makes guesses, tests give proof, humans decide."

In very simple words

- Usually people ask AI to **write** code. I asked it to **break** code.
- It found **some real bugs**, got **some things wrong**, and **missed the biggest one**.
- **AI did help**. It's like a fast junior tester: useful, but you check its work.

CHAPTER 31

Latest results and how efficient the AI was

All runs used `qwen2.5-coder:7b` via Ollama on a 16 GB MacBook Air, on 17–18 Sep 2026. Output unedited and committed in `ai/`.

Run	Time	Tests	Red	Real bugs	Notes
Prompt 2 · bug hunting the demo run	91 s	8 (+1 rejected)	5	3	Bad token → 500 · fake admin token · Bob reads Alice's order. 2 false alarms.
Prompt 1 · input checks	≈105 s 1 retry	10	2	0	8 confirmed the API rejects bad input types. 2 reds were 401 for bad logins: reasonable, not bugs.
Prompt 3 · money logic	20 s + 72 s	1 + 4	1 + 0	0	First run invented <code>/buy_ticket</code> (prompt lacked endpoints). Second: 4 near-identical "buy 11" tests.

The efficiency numbers

60%

Prompt 2 precision: when it said "bug", it was right 3 times out of 5.
Quote this one.

30%

Bugs found: 3 of the 10 planted bugs.

43%

All prompts together: 3 real out of 7 reds. Only if asked.

How to talk about the numbers

SAY

- "Three real bugs my own tests missed, in 90 seconds, on a laptop, for free."
- "When it said 'bug', it was right 60% of the time. That's why every red line needs a human."
- "One run, one small model, one small API."

DON'T SAY

- "AI finds 30% of bugs" (as a general fact)
- "A bigger model would find everything"
- "AI is not efficient"

Is 3 out of 10 bad?

No. Your original happy-path tests found **0** of these bugs. The AI found 3 in about 90 seconds, and the prompt focused on security, not number limits or money. It's **useful and fast, but incomplete**, which is exactly why the engineering and the human review matter.

CHAPTER 32

Concepts you must be able to explain

One line each, in plain words, with the example from your talk. If you can say these, you can answer most questions.

Concept	Plain meaning	Example in your talk
API	How one program asks another for something	ConfBadge: login, search, order
Status code	The API's verdict: 2xx worked, 4xx you asked wrong, 5xx server broke	Bad token got 500 instead of 401
Happy path	Testing only the normal way a feature is used	"Buy 2 tickets" works fine
Adversarial testing	Deliberately trying to break software	The whole talk
API vs unit test	Unit test = one function; API test = the whole running app from outside	Your tests send real HTTP requests
Test oracle	The rule for what the <i>correct</i> answer is	"expect_status = what a CORRECT API returns"
Boundary testing	Testing at, below and above a limit	Max 10 per order: 10 is rejected (bug)
Authentication	Proving who you are	Fake token gives admin access
Authorisation / IDOR	Checking you're allowed to see <i>this</i> item	Bob reads Alice's order by changing the ID
SQL injection	Input that changes a database query	Search text reveals a hidden ticket
Replay	Sending the same valid request again	SPEAKER100 used forever
Chained bugs	Two small bugs making one big one	Coupon + credits = money loop
Race condition	Timing bug when requests happen at once	20 buyers, 5 seats, all accepted

AI and engineering concepts

Concept	Plain meaning	Example in your talk
LLM	An AI that predicts text from patterns in what it learned	qwen2.5-coder:7b
Local model / Ollama	The AI runs on your laptop: free, private, offline	No internet needed on stage
7B	7 billion parameters: small enough for a laptop, less capable than big online models	Why it took 90 s and made mistakes
Prompt engineering	Writing instructions that get useful output	"Assume the developer was rushed..."
Structured output	Making the AI answer in a fixed format (JSON)	Every idea becomes a runnable test
Validation	Automatically rejecting broken AI output	1 case rejected (raw token as user)
Hallucination	AI inventing something that isn't real	Made-up <code>/buy_ticket</code> endpoint
False positive	A "bug" that isn't a bug	TC-006 expected a 500; TC-009 mixed up coupons
Precision	Of the things flagged, how many were real	3 of 5 = 60%
Human in the loop	A person reviews AI output before acting on it	You triage every red line
Triage	Deciding if a finding is real and how important	Slide 15: four questions
Test isolation	Resetting data before each test	Same result every run
CI/CD	Automatically running tests on every code change	"Run them as a separate job"

TECH STACK IN ONE BREATH

"Python, FastAPI and SQLite for the API; a local model through Ollama for the AI; httpx and pytest to run the tests. Everything runs on this laptop, with no internet and no paid APIs."

CHAPTER 33

The 30-minute plan (26 slides)

About 11 minutes in the terminal. Checkpoints in bold: if you're behind at one, use the cut listed.

Clock	Slides	Section	Mode · cut if behind
00:00–01:30	1–2	Title · agenda	Slides. Say the theme line.
01:30–05:30	3–5	The problem · API basics · ConfBadge	Slides. Behind? Shorten slide 4 to 30 s.
05:30–06:30	6	"This works" curl	Terminal
06:30–10:15	7–10	The idea · prompt rules · test case · beyond the prompt	Slides
10:15	11–12	Demo 1 & 2: output, run	Terminal
12:30–15:45	13–15	3 real bugs · 2 false alarms · triage	Slides. Behind? Skip slide 15.
15:45	16–17	Money loop ★ wow moment	Terminal
18:30	18–19	Race · student ticket	Terminal . Behind? Skip student demo.
21:15–24:15	20–22	Engineering caught · numbers · verdict	Slides. Never skip 21.
24:15	23–24	Key sentence · Monday actions	Slides
26:00–30:00	25–26	Thank you · feedback QR · Q&A	Stay on 26 while people scan

TERMINAL SETUP

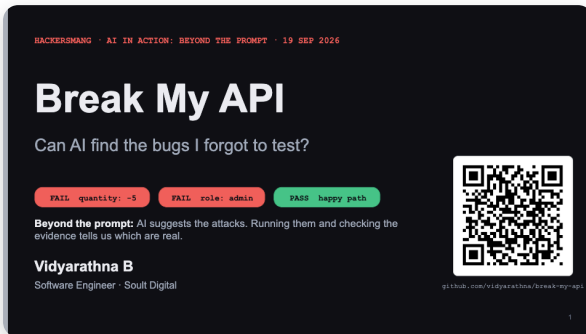
- Tab A: API running · Tab B: commands
- Light theme, font 22pt+
- `export PS1="$ "`

NEVER ON STAGE

- Generating with the AI (90 s)
- Editing code
- Debugging for more than 20 seconds

Slide by slide: what to explain

For `slides/Break-My-API.pptx` (26 slides, dark theme). Each block: the slide, what to explain, the line to say, and a likely question.



SLIDE 1

Break My API

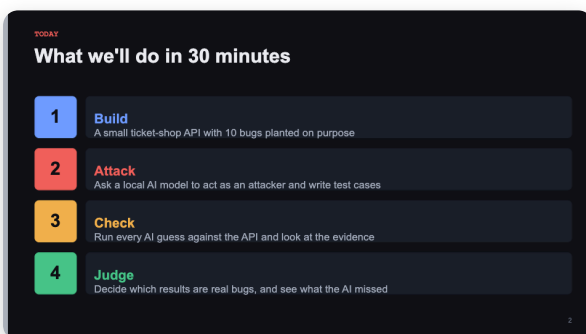
Slides · 00:00

EXPLAIN

- Your name and the event theme: **AI in Action: Beyond the Prompt**.
- The red/green tags preview the talk: failures AI found vs the happy path.
- QR code = the repo; people can follow along.

"Today's theme is 'beyond the prompt'. My talk is a small, real example: the prompt got AI to suggest attacks, but the engineering around it told me which ones were true."

If asked: "Is the code available?" → Yes, everything is in the repo on the QR code: API, prompts, AI output, runner, this guide.



SLIDE 2

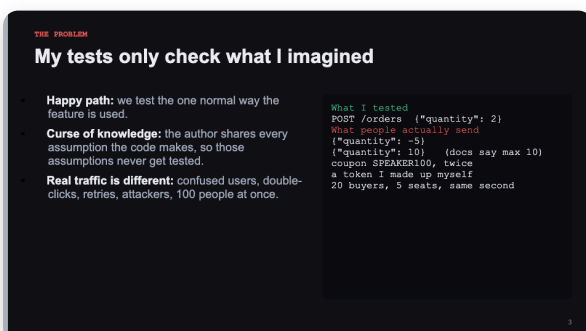
What we'll do in 30 minutes

Slides · 00:45

EXPLAIN

- Four steps: **Build** → **Attack** → **Check** → **Judge**.
- Build = small API with 10 planted bugs. Attack = AI writes tests.
- Check = run them. Judge = decide what's real.

"Most of us use AI to write code. Today I'll use it to break code, and then check whether it was right."



SLIDE 3

My tests only check what I imagined

Slides · 01:30

EXPLAIN

- **Happy path:** we test the normal way it's used.
- **Curse of knowledge:** the author shares the code's assumptions, so never tests them.
- Right box: what real users and attackers actually send.

"When I write 'quantity is a number', my brain fills in 'a positive number'. I test with 2, it passes. Nobody tests minus five."

If asked: "Isn't this just negative testing?" → Yes, and that's the point: AI helps generate negative tests we don't think of.

THEORY · 1 MINUTE

What an API is, and how it answers

- **API:** the way one program asks another for something. Like a waiter between you and the kitchen.
- **Request:** a method (GET = read, POST = create), a path, headers (who you are), and a JSON body.
- **Response:** a status code plus JSON data.

2xx It worked
 200 OK, 201 Created.
 Dangerous when it's a lie: "201" for minus five tickets.

4xx You asked wrong
 400, 401, 403, 404. The API refused politely. This is what we want for bad input.

5xx The server broke
 500. If bad user input causes a 500, that is almost always a bug.

SLIDE 4

What an API is, and how it answers

Slides · 03:00 · keep short

EXPLAIN

- API = a waiter between you and the kitchen.
- Request: method, path, headers, JSON body. Response: status + JSON.
- **2xx** worked · **4xx** you asked wrong · **5xx** server broke.

"Most tests today ask one question: when I send something wrong, do I get a polite 4xx?"

THE API UNDER TEST

ConfBadge: a small conference ticket shop

Endpoints
 POST /auth/login log in ~ token
 GET /tickets?q search tickets
 POST /orders buy tickets
 GET /orders/{id} read one order
 Python · FastAPI · SQLite

1. Everyone has a prepaid credit balance.
 2. Buying a ticket takes the price from your balance.
 3. Every purchase gives 10% back as credits. Credits are money.
 4. SPEAKER100 coupon = 100% off, single use.
 5. Maximum 10 tickets per order.

Tickets: GENERAL 2000 · STUDENT 500 · WORKSHOP 3500 (5 seats) · STAFF (hidden)

SLIDE 5

ConfBadge: a small ticket shop

Slides · 04:00

EXPLAIN

- 4 endpoints: login, search, order, read order.
- 5 rules. **Pause on rule 3: credits are money.**
- Tickets, including a hidden STAFF ticket.

"Hold on to rule three, credits are money. It comes back later."

If asked: "Why build a fake API?" → So the bugs are known and safe to show. Everything resets in milliseconds.

LIVE · TERMINAL

This works. I tested it.

```
$ curl -X POST localhost:8000/orders \
  -H "authorization: Bearer $TOKEN" \
  -d '{"ticket_id": 1, "quantity": 2}'
201 Created
{"total_charged": 4000.0,
 "credits_earned": 400.0,
 "credit_balance": 1400.0 }
```

Everything is correct
 2 tickets × 2000 = 4000 charged.
 10% back = 400 credits.
 5000 - 4000 + 400 = 1400.
 Every test I wrote passes. That's the problem.

SLIDE 6

This works. I tested it.

Terminal · 05:30

EXPLAIN

- Run the normal order live.
- 2 tickets: 4000 charged, 400 back, balance 1400.
- Everything is correct.

"Correct. I wrote this, I tested it, the tests pass. That's the problem."

THE IDEA

Don't ask AI to write code. Ask it to break it.

1 · AI model Guesses what might break. Fast and cheap. Sometimes wrong.	2 · Test runner Sends every guess to the real API and records what came back. Dumb on purpose.	3 · Human Decides which red lines are real bugs. Knows the business rules and the priorities.
---	---	--

AI generates hypotheses. Execution provides evidence. **Humans decide what is a defect.**

SLIDE 7**Don't ask AI to write code. Ask it to break it.**

Slides · 06:30

EXPLAIN

- Three roles: **AI model** guesses, **test runner** checks, **human** decides.
- Each fails differently, so you need all three.
- Bottom line = the key sentence (first time).

"The model is fast and sometimes wrong. The runner is dumb on purpose. The human knows the business."

If asked: "Why not just trust the AI?" → Because in my run 2 of its 5 'bugs' were wrong. The runner and I caught them.

HOW TO PROMPT AN ATTACKER

Four rules that turn "write tests" into attacks

1 · Ask it to break, not to test "Assume the developer was rushed and the non-obvious validation is missing."	2 · Ask for the correct answer "expect_status = what a CORRECT API returns, not what this one returns."
3 · Give it real context The endpoint list, the business rules, a real login token.	4 · Force structured output JSON with fixed fields, so every guess can be run as a test.

SLIDE 8**Four rules that turn 'write tests' into attacks**

Slides · 07:30

EXPLAIN

- 1 Ask it to break. 2 Ask for the **correct** answer.
- 3 Give real context (endpoints, rules, token). 4 Force JSON output.
- Rule 2 is the most important.

"Without rule two, the AI writes down what the API does now, and every test passes: a green suite that certifies your bugs."

If asked: "What prompt did you use?" → It's in ai/prompts.md; I can print it with --print-prompt.

ONE AI-GENERATED TEST CASE

Every guess becomes something we can run

```

{
  "method": "GET",
  "endpoint": "/orders/1",
  "auth": "bob",
  "expect_status": [403],
  "objective": "Read another user's order",
  "reason": "A2: assumes callers have permission",
  "severity": "high"
}

```

Executed method, endpoint, auth, payload, expected status. These hit the real API.	Read by humans objective, reason, severity. Never executed. A wrong reason costs a sentence, not a false result.
--	--

SLIDE 9**Every guess becomes something we can run**

Slides · 08:30

EXPLAIN

- One real AI test case in JSON.
- Orange fields run:** method, endpoint, auth, expected status.
- Grey fields** are for humans: objective, reason, severity.

"If the AI invents an endpoint, it fails loudly with a 404. A wrong reason costs a sentence, not a false result."

If asked: "Is this a unit test?" → No, an API test: it tests the whole running app from outside, with real HTTP requests.

BEYOND THE PROMPT

The prompt is the easy part. Engineering makes it useful.

1 Prompt attacker: instructions + real API context	2 AI model writes test ideas as JSON	3 Validate broken AI output rejected automatically	4 Run every idea sent to the real API	5 Evidence PASS / FAIL, with the real response	6 Human decides which reds are real bugs
--	---	--	--	---	---

- Structured output:** JSON with fixed fields, so every AI idea becomes a runnable test.
- Validation:** malformed ideas are rejected before they run (1 of 9 today).
- Execution:** results come from the real API, not from what the AI claims.
- Human review:** the AI's idea of "correct" is checked, like any production AI system.

10

SLIDE 10

Beyond the prompt: engineering makes it useful

Slides · 09:15 · theme slide

EXPLAIN

- The pipeline: Prompt → AI → Validate → Run → Evidence → Human.
- Structured output, validation, execution, human review.
- This is the event theme, made concrete.**

"The model only gives ideas. Everything around it is what turns ideas into results you can trust."

If asked: "What does 'beyond the prompt' mean?" → The prompt is step one. The system around it (checking, running, reviewing) is what makes AI reliable.

DEMO 1 · TERMINAL

What the AI wrote

```
$ python -m json.tool ai/run_prompt2.json | head -40
```

- Model:** qwen2.5-coder:7b running locally with Ollama. No internet, no paid API.
- Time:** 91 seconds to generate. Too slow to do live, so this is last night's unedited output.
- Output:** 9 test cases. 1 was rejected automatically (it put a raw token where a user name belongs). 8 run.
- Its 5 assumptions:** tokens are valid · callers have permission · input is clean · numbers are in range · things are used once.

11

SLIDE 11

Demo 1: What the AI wrote

Terminal · 10:15

EXPLAIN

- Show `ai/run_prompt2.json` (don't generate live).
- Model qwen2.5-coder:7b, local, 91 seconds.
- 9 cases, 1 rejected automatically. Read the 5 assumptions.

"I ran this prompt last night on a local model: no internet, no paid API. This is the unedited output."

If asked: "Why not generate live?" → It takes 90 seconds and gives different results each time. The saved, unedited output is more honest.

DEMO 2 · TERMINAL

Run every guess against the API

```
$ python -m tests.runner --cases ai/run_prompt2.json
```

- Each case:** reset the database, send the request, compare the status with what the AI said is correct.
- PASS** means this guess was not confirmed. It does not mean "no bug".
- FAIL** means a candidate defect. The AI's idea of "correct" can be wrong.

Result: 8 cases · 3 PASS · 5 FAIL

12

SLIDE 12

Demo 2: Run every guess against the API

Terminal · 11:30

EXPLAIN

- Run the runner; pause while it prints.
- PASS = guess not confirmed (not 'no bug'). FAIL = candidate defect.
- Result: 8 cases, 3 PASS, 5 FAIL.

"Eight guesses, five came back red. Now let's decide which five are real."

3 REAL BUGS

What the AI got right

TC-002	Broken token → server crash Expected 401, got 500 with Python's internal error text.	Bad input should get a polite 4xx. A 500 leaks details and triggers false alarms in monitoring.
TC-004	Fake token → admin access Expected 401, got 200.	The token is just base64 of "user_id:role". Anyone can write "9:admin" and encode it. Not signed.
TC-005	Bob reads Alice's order Expected 403, got 200.	IDOR: the API checks you're logged in, not that the order is yours. Change /orders/1 → someone else's data.

13

SLIDE 13

3 real bugs: what the AI got right

Slides · 12:30

EXPLAIN

- **TC-002** broken token → 500 with Python error.
- **TC-004** fake token → admin access (token isn't signed).
- **TC-005** Bob reads Alice's order (IDOR).

"Read expected, then got, for each. These are real bugs my own tests had missed."

If asked: "How would you fix the token bug?" → Sign tokens with a secret (JWT) and return 401 for anything invalid.

2 FALSE ALARMS

What the AI got confidently wrong

TC-006 · SQL injection The AI said a correct API should return 500 for an injection attempt. Wrong: a crash is never the correct answer. The API returned 200 with normal results. The test went red for the wrong reason.	TC-009 · EARLYBIRD coupon The AI tested EARLYBIRD as a "use once" coupon and expected a rejection. Wrong: the docs say EARLYBIRD is unlimited. The single-use coupon is SPEAKER100. It mixed them up.
---	--

5 red lines → 3 real bugs. **Every red line needs a human to check it.**

14

SLIDE 14

2 false alarms: confidently wrong

Slides · 14:00 · credibility moment

EXPLAIN

- **TC-006:** AI said a correct API returns 500 for SQL injection. Wrong.
- **TC-009:** AI treated unlimited EARLYBIRD as single-use. Wrong.
- Both looked like findings.

"Both are confident. Both are wrong. That's why the runner says candidate defects, and why the last step is a person."

If asked: "How do you prevent hallucinations?" → I don't; I make them cheap. Everything is executed and a human checks every red.

THEORY · TRIAGE

Four questions for every red line

- 1 Does the endpoint exist?**
If not, the AI hallucinated. In another run it invented /buy_ticket, which returned 404.
- 2 Is the AI's "correct" really correct?**
TC-006 expected a 500. TC-009 misread the coupon rules. Check the docs.
- 3 Does it happen again after a reset?**
If not, it's a test setup problem, not a bug.
- 4 Does it matter to users or money?**
Real but minor goes to the backlog. Real and serious gets fixed first.

15

SLIDE 15

Four questions for every red line

Slides · 15:00 · skip if behind

EXPLAIN

- 1 Does the endpoint exist? 2 Is the AI's 'correct' right?
- 3 Does it repeat after a reset? 4 Does it matter to users or money?
- A person who knows the system answers these in a minute.

"The AI can't answer these reliably. Someone who knows the system can."

DEMO 3 · THE MONEY LOOP

Two small bugs, one big problem

```
$ grep -n -E 'le*(total|credits_earned)' app/main.py
139: total = price * quantity * (100 - discount)
151: credits_earned = price * quantity * 100
```

Bug 1 · coupon reuse

SPEAKER100 is documented as single use, but nothing records that it was used. It works every time.

Bug 2 · credits from list price

You earn 10% of the full price, not of what you paid. With a 100% coupon you pay 0 and still earn credits.

16

SLIDE 16

Money loop: two small bugs, one big problem

Terminal · 15:45 · ★

EXPLAIN

- Show two code lines: 139 what you pay, 151 what you earn.
- Earn from **list price**, not paid price.
- Plus SPEAKER100 reusable. Ask the room to guess.

"Five rounds, starting at 5,000. Where does the balance end?"

DEMO 3 · TERMINAL

Nothing was invalid. The balance still grew.

```
$ python demo/money_loop.py
```

start	order 1	order 2	order 3	order 4	order 5
5,000	6,800	8,600	10,400	12,200	14,000

- **Paid:** 0.00 on every order. Earned: 1,800 every time. +9,000 credits from nothing.
- **Every request returned 201.** Nothing looked wrong. A dashboard would show a happy customer.
- **The AI didn't find this.** I asked it to attack the money rules: it tested "buy 11 tickets", four times.

17

SLIDE 17

Nothing was invalid. The balance still grew.

Terminal · 17:00 · wow moment

EXPLAIN

- Run money_loop.py and stay quiet.
- 5,000 → 14,000, paid 0.00 every time, all 201s.
- **The AI didn't find this. I did.**

"Every request returned 201. Nothing looked wrong. I found this because I know credits are money here."

If asked: "Would a bigger AI find it?" → Maybe, I haven't tested one. It still needs the rule 'credits are money' written down.

DEMO 4 · RACE CONDITION

20 buyers, 5 seats, the same second

```
$ python demo/race.py
orders accepted (201): 20
orders rejected (409): 0
seats remaining after: 3
*** oversold by 15 seats ***
```

What happens: each request reads "5 seats", waits for payment, then writes "4". They all read before anyone writes.

Why tests miss it: every single request is valid. The bug is in the timing between requests.

Why the AI missed it: its test format is one request, one expected answer. It can't say "twenty at once".

Fix: one atomic database update: only subtract if stock is still enough.

18

SLIDE 18

Race: 20 buyers, 5 seats

Terminal · 18:30

EXPLAIN

- Run race.py: 20 accepted, 0 rejected, oversold by 15.
- Everyone reads '5 seats' before anyone writes '4'.
- The test format is one request, one answer: it can't say 'twenty at once'.

"Every request is valid. The bug is in the timing between them."

If asked: "How do you fix it?" → One atomic update: subtract only if stock is still enough, in a single database statement.

DEMO 4 · THE UNWRITTEN RULE

Is this a bug? You can't tell.

```
$ curl ... POST /orders ("ticket_id": 2) (as alice@corp.example)
201 Created
{ "ticket": "STUDENT-2026", "total_charged": 500.0 }
```

- **The rule:** student price needs a verified student email. That rule is in a pricing document, not in the code.
- **The AI:** might guess to try it, but it can't know the 201 is wrong. Neither can a new developer reading the code.
- **The lesson:** write business rules where a reader can find them: API docs, schema, tests.

19

SLIDE 19

Is this a bug? You can't tell.

Terminal · 20:00 · skip if behind

EXPLAIN

- **Reset first.** Buy STUDENT ticket as alice@corp.example → 201.
- The rule is in a pricing document, not in the code.
- Lesson: write business rules down.

"Neither the AI nor a new developer can know this is wrong. The information isn't there."

BEYOND THE PROMPT

What the engineering caught

Execution caught the mistakes Two confident but wrong results (a 500 called "correct", mixed-up coupons). Running them against the real API exposed both.	A better harness handles timing The race condition needs "twenty requests at once". Add that to the test format and it becomes testable.	Written rules become testable The student-pricing rule lived only in a document. Write it into the API docs and the AI can test it too.
---	--	---

The money loop needed business knowledge: a human connected the two bugs.

20

SLIDE 20

What the engineering caught

Slides · 21:15 · positive framing

EXPLAIN

- Execution caught the AI's mistakes.
- A better harness can test timing.
- Written rules become testable. Money loop needed business knowledge.

"Each of these is handled by engineering around the model, not a better prompt."

THE HONEST NUMBERS

Real run: qwen2.5-coder:7b, local, unedited

8 tests run	5 came back red	3 real bugs	2 false alarms
----------------	--------------------	----------------	-------------------

- 0 findings on the money bugs
- 1 case rejected as malformed before running
- 91 seconds to generate on a laptop
- 60% of red lines were real bugs (3 of 5)
- 2nd prompt (input checks): 10 more tests. 8 confirmed the API rejects bad input types, 2 arguable, 0 new bugs.

21

SLIDE 21

The honest numbers

Slides · 22:15 · never skip

EXPLAIN

- 8 run · 5 red · 3 real · 2 false alarms.
- 60% of reds were real. 91 seconds.
- 2nd prompt: 10 tests, 8 confirmed checks work, 0 new bugs.

"Five red, three real, two confidently wrong, zero on the expensive bug."

If asked: "Is 3 out of 10 good?" → My original tests found zero. It found three in 90 seconds. Useful and fast, but incomplete.

SO, CAN AI FIND THE BUGS I FORGOT TO TEST?

AI + engineering: fast, useful, trustworthy.

What the AI brought	What engineering + people brought
• 3 real bugs my own tests missed • Attack ideas in about 90 seconds • Common attacks: fake tokens, other users' data, crashes • Checks that confirm the basics work • A reason for every test	• Structured output so ideas can run • Automatic rejection of broken output • Real evidence from the real API • Judgement on what "correct" means • Business knowledge: the money loop

22

SLIDE 22

AI + engineering: fast, useful, trustworthy

Slides · 23:15 · answers the title

EXPLAIN

- Left: what AI brought (3 real bugs, speed, reasons).
- Right: what engineering + people brought.
- So: yes, AI can find bugs I forgot, with engineering around it.

"Yes: three real bugs my tests missed, in 90 seconds, on a laptop, for free. The engineering makes those results trustworthy."

AI generates hypotheses.

Execution provides evidence.

Humans decide what is a defect.

The prompt is the easy part. The engineering around it makes AI useful.

23

SLIDE 23

The key sentence

Slides · 24:15

EXPLAIN

- Say it slowly, word for word.
- Then the theme line underneath.
- Pause between lines.

"AI generates hypotheses. Execution provides evidence. Humans decide what is a defect. The prompt is the easy part."

TRY IT ON MONDAY

Four things you can do at work

- 1 Pick one important endpoint**
Orders, payments, permissions. Run a test environment you can reset.
- 2 Save the AI's test cases in the repo**
Review them like code. Don't regenerate them on every build.
- 3 Run them in CI as a separate job**
Non-blocking until a human has checked the red lines.
- 4 Write your business rules down**
Every rule a reader can find is a rule a tool can test.

24

SLIDE 24

Try it on Monday

Slides · 25:00

EXPLAIN

- 1 Pick one important endpoint. 2 Save AI tests in the repo.
- 3 Run them in CI as a separate job. 4 Write business rules down.
- End on point 4.

"If you do one thing: write down the rule your team only knows from a meeting."

If asked: "Can this run in CI?" → Yes: generate occasionally, commit the tests, run them in a separate non-blocking job.

Thank you. Break it yourself.

- **Everything runs locally:** Python, FastAPI, SQLite, Ollama. No paid APIs.
- **In the repo:** the API, prompts, real AI output, test runner, demos, and a full PDF guide.
- **Questions?**



github.com/vidyarthina/break-my-api

Vidyarthina B · Soult Digital

25

SLIDE 25**Thank you. Break it yourself.**

Slides · 26:00

EXPLAIN

- Everything runs locally: Python, FastAPI, SQLite, Ollama.
- Repo has the API, prompts, AI output, runner, demos, guide.
- Invite questions.

"Everything's in that repo. Break it yourself. Questions?"

FEEDBACK

How was this talk?

- **Scan the QR code** to share your feedback.
- **It takes under a minute.**
- **Tell me:** what worked, what didn't, and what you'd like to see next.



Break My API: Can AI Find the Bugs I Forgot to Test?

Thank you for listening!

26

SLIDE 26**How was this talk? (feedback QR)**

Slides · 26:30 → Q&A

EXPLAIN

- Leave this slide up during Q&A.
- Ask people to scan and share feedback.
- It takes under a minute.

"It takes under a minute, and it genuinely helps me improve."

CHAPTER 35

Demo runbook and backup plan

Every command in order. Print this page. Run everything from `~/Downloads/break-my-api`.

```
# BEFORE · tab A (leave running, no --reload on stage)
.venv/bin/uvicorn app.main:app --port 8000
# tab B
curl -s -X POST localhost:8000/_reset
TOKEN=$(curl -s -X POST localhost:8000/auth/login -H 'content-type: application/json' \
  -d '{"email":"alice@corp.example","password":"alice123"}' | jq -r .access_token)

# SLIDE 6 · the normal order
curl -s -X POST localhost:8000/orders -H "authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"ticket_id":1,"quantity":2}' | jq
# SLIDE 11 · what the AI wrote
.venv/bin/python -m json.tool ai/run_prompt2.json | head -40
# SLIDE 12 · run every guess → 3 matched, 5 did not
.venv/bin/python -m tests.runner --cases ai/run_prompt2.json
# SLIDE 16 · the two code lines
grep -n -E '^\s+(total|credits_earned) = ' app/main.py
# SLIDE 17 · money loop → 5,000 → 14,000
.venv/bin/python demo/money_loop.py
# SLIDE 18 · race → 20 accepted, oversold by 15
.venv/bin/python demo/race.py
# SLIDE 19 · reset FIRST, then the student ticket → 201, 500.0
curl -s -X POST localhost:8000/_reset
curl -s -X POST localhost:8000/orders -H "authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"ticket_id":2,"quantity":1}' | jq
```

If this happens	Do this (never debug more than 20 seconds)
API not responding	Tab A: Ctrl+C, start it again, then <code>/_reset</code> .
Port 8000 already in use	<code>lsof -ti:8000 xargs kill</code> then start again.
Strange numbers (negative balance)	<code>curl -s -X POST localhost:8000/_reset</code> . Leftover data from a previous demo.
<code>python: command not found</code>	Use <code>.venv/bin/python</code> , or <code>source .venv/bin/activate</code> .
Terminal or projector fails	Show <code>demo/screenshots/</code> and narrate the same points.
Internet down	Nothing: the demo needs no internet. Say so; it lands well.
Running 3+ minutes late	Skip slide 15 and the student demo (slide 19). Never skip slide 21.

CHAPTER 36

Q&A: what people will ask

Repeat the question, answer in two or three sentences, then stop. "I haven't tested that; here's how I would" is a strong answer.

1. So did AI actually help?

Yes. It found 3 real bugs my own tests missed, in about 90 seconds. It also got 2 wrong and missed the money bug, so I check its work, like a fast junior tester.

2. Isn't 3 out of 10 bugs bad?

My original happy-path tests found zero. The prompt focused on security, not limits or money. Useful and fast, but incomplete, which is why the engineering and review matter.

3. Which model did you use, and why?

qwen2.5-coder:7b through Ollama. It runs on my laptop, it's free, it keeps the API private, it works offline, and it's trained for code. The trade-off: slower and less accurate than big online models.

4. Would ChatGPT or Claude do better?

Possibly; bigger models usually reason better. I only measured this one, so I won't claim a number. The engineering around it would stay the same.

5. Is this like unit testing?

No, it's API testing: the whole running app, tested from outside with real HTTP requests. It uses pytest, so it looks similar, but it tests a different level.

6. Isn't this just fuzz testing?

A fuzzer sends random inputs by volume. The AI picks inputs from meaning, like "max 10 per order", and explains why. They find different bugs; use both.

7. How does this relate to "beyond the prompt"?

The prompt only produced ideas. Structured output, validation, running the tests and human review turned them into trustworthy results. That's the engineering part of the theme.

8. How do you stop hallucinations?

I don't stop them; I make them cheap. Only method, endpoint, auth and payload run. An invented endpoint just returns 404, and broken output is rejected automatically.

9. How do you know the AI's tests are correct?

I don't assume they are. A FAIL is only a candidate. I ask four triage questions: does it exist, is its "correct" right, does it repeat, does it matter.

10. Does this replace QA engineers?

No. It moves effort from writing test cases to judging them. The false alarms and the money bug both needed someone who understands the system.

Q&A, continued

11. Why didn't you generate the tests live?

It takes about 90 seconds and gives different results each time. The saved output is unedited and committed to the repo, so it's more honest than a lucky live run.

12. Isn't the demo rigged? You planted the bugs.

The bugs are planted, and I say so. The AI's output isn't rigged: it's raw, mistakes included, and it missed most of what I planted.

13. Can AI find security vulnerabilities?

It found a forgeable token, an access-control bug (IDOR) and a crash that leaks internals. It's a useful first pass, not a replacement for a security review.

14. Why did it miss the money bug?

It needs two bugs connected, plus knowing that credits are real money. That's business knowledge and multi-step reasoning, the hardest kind for a small model.

15. Why can't it find the race condition?

My test format is one request, one expected answer. A race needs "twenty requests at once". That's a limit of the harness, which engineering can fix.

16. Did you try other prompts?

Yes. An input-checks prompt gave 10 more tests: 8 confirmed the basics work, 0 new bugs. A money-logic prompt first invented an endpoint, then repeated one idea.

17. How much did the prompt matter?

The most important line is "expect what a correct API returns". Without it, tests just copy current behaviour and pass. Adding the real endpoint list fixed the invented endpoint.

18. Can I give it the bug names to get better results?

That's cheating: you'd be telling it the answer. General instructions like "test every limit" or "repeat anything single-use" are fair.

19. Is it safe to send my API to an AI?

Here nothing leaves the laptop. With online models, send only test data and test tokens, never real customer data or production secrets.

20. Can this run in CI/CD?

Yes. Generate occasionally, review and commit the tests, and run them in a separate non-blocking CI job. Promote confirmed bugs into normal tests.

Q&A, harder questions

21. Can it work with Postman or Playwright?

Yes. The tests are plain JSON: method, endpoint, payload, expected status. Anything that can send HTTP can run them.

22. How would you use this at work?

Pick one important endpoint, run it against a test environment, commit the AI's tests, triage the reds once, and keep the real ones as regression tests.

23. How do you measure if it's worth it?

How many reds are real (60% here), bugs your suite missed, minutes spent per red, and bugs that later escape to production.

24. Why 7B and not a bigger local model?

It fits a 16 GB laptop and responds in about 90 seconds. Bigger models need more memory and time. I chose reliability on stage.

25. Could the AI fix the bugs too?

For simple ones, likely yes. For the race condition, a fix can look right and still fail. AI fixes need the same review as AI findings.

26. What's the biggest lesson for you?

Write business rules down. The student-pricing rule lived only in a document, so no tool, and no new developer, could test it.

27. What would you improve next?

Let the test format describe sequences and "run these together", run prompts several times and merge, and try a larger model.

28. What language and tools is this?

Python, FastAPI and SQLite for the API; Ollama for the model; httpx and pytest for the runner. All local, no paid APIs.

IF YOU DON'T KNOW

"Good question. I haven't tested that. Here's how I would..." Then describe the experiment. That's a strong answer, not a weak one.

CHAPTER 37

Checklists for tonight and tomorrow

Tonight

- ☐ Open the deck in Keynote; click through all 26 slides
- ☐ One full timed run-through, standing
- ☐ Practise the 1-minute pitch out loud twice
- ☐ Read the Q&A pages once
- ☐ Record a short backup video of the 4 demos
- ☐ Charge laptop; pack charger + HDMI/USB-C adapter
- ☐ Sleep. No code changes after tonight.

Morning

- ☐ Laptop charged; notifications off; sleep disabled
- ☐ Start API: `.venv/bin/uvicorn app.main:app --port 8000`
- ☐ Run the runner once: 3 matched, 5 did not
- ☐ Deck open on slide 1
- ☐ Print or open the runbook page

30 minutes before

- ☐ Connect to the projector; check the terminal is readable
- ☐ Terminal: light theme, font 22pt+, `PS1="$ "`
- ☐ `/_reset`, then log in (the TOKEN line)
- ☐ Type the slide 6 curl, don't run it
- ☐ Feedback QR tested with your phone

5 minutes before

- ☐ `/_reset` again
- ☐ Water nearby
- ☐ Say your first sentence once, quietly
- ☐ Breathe. You know this better than anyone in the room.

During the talk

- ☐ Say the theme line on slide 1
- ☐ Zoom before running a command
- ☐ Pause 2 seconds after output appears
- ☐ Read "expected", then "got"
- ☐ Reset before the student demo
- ☐ Clock checks: 10:15, 15:45, 18:30, 24:15
- ☐ Key sentence slowly, once
- ☐ End on feedback slide for Q&A

BEYOND THE PROMPT

**AI generates hypotheses.
Execution provides evidence.
Humans decide what is a defect.**

The prompt is the easy part. The engineering around it is what makes AI useful.